The DSA Notebook

GRINDING 2.0

Pattern-wise breakdown

Arrays & Sorting Fundamentals

- Two Sum
- Best Time to Buy & Sell Stock
- Move Zeros
- Majority Element
- Product of Array Except Self
- Rotate Array
- Next Permutation

Hashing

- Contains Duplicate
- Valid Anagram
- Group Anagrams
- Top K Frequent Elements
- Longest Consecutive Sequence
- Subarray Sum Equals K

Prefix Sum

- Range Sum Query –Immutable
- Find Pivot Index
- Subarray Sum Equals K
- Continuous Subarray Sum(divisible by K)

Two Pointers

- Valid Palindrome
- Two Sum II – Input array is Sorted
- 3 Sum
- Container with Most Water
- Sort Colors
- Trapping Rain Water

Pattern-wise breakdown

Sliding Window

- Maximum Sum of Distinct Subarrays With Length K
- Longest Substring Without Repeating Characters
- Longest Repeating Character Replacement
- Permutation in String
- Sliding Window Maximum

Binary Search

- Binary Search
- Search Insert Position
- Find First and Last Position of Element
- Search in Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Search a 2D Matrix
- Koko Eating Bananas
- Median of Two Sorted Arrays

Recurssion

- Reverse String
- Fibonacci Numbers (Recursive + Memoized)
- Pow(x,n)
- Merge Sort
- Quick Sort
- Escape Maze
- Subsequence

Backtracking

- Subsets
- Permutations
- Generate Parentheses
- Combination Sum
- Word Search
- N-Queens

Pattern-wise breakdown

- Linked List
  - Reverse Linked List
  - Merge Two Sorted Lists
  - Linked List Cycle
  - Add Two Numbers
  - Remove Nth Node From End of List
  - Copy List with Random Pointer
  - LRU Cache

Topics

- Arrays
- Binary Search
- Strings
- Linked List
- Recursion
- Bit Manipulation
- Stack and Queues
- Sliding Window & Two Pointer
- Heaps
- Greedy Algorithms
- Binary Trees
- Binary Search Tree
- Graphs
- Dynamic Programming
- Tries
- Strings (Advance)

Arrays

- Check if Array Is Sorted and Rotated
- Remove Duplicates from Sorted Array
- Rotate Array
- Move Zeroes
- Missing Number
- Max Consecutive Ones
- Single Number
- Two Sum
- Sort Colours
- Majority Element
- Maximum Subarray
- Best Time to Buy and Sell Stock
- Rearrange Array Elements by Sign
- Next Permutation
- Leaders in an Array
- Longest Consecutive Sequence
- Set Matrix Zeroes
- Rotate Image
- Spiral Matrix
- Subarray Sum Equals K
- 3 Sum
- 4 Sum
- Largest subarray with 0 sum
- Count Subarrays with given XOR
- Merge Intervals
- Merge Sorted Array
- Merge Without Extra Space
- Count Inversions
- Reverse Pairs
- Maximum Product Subarray
- Frequency of the Most Frequent Element
- Longest Subarray with given Sum K (non-negative)
- Longest Subarray with given Sum K
- Product of Array Except Self
- Top K Frequent Elements

Binary Search

- Binary Search
- Search Insert Position
- Implement Lower Bound
- Implement Upper Bound
- Find First & Last Position of Element
- Search in Rotated Sorted Array
- Search in Rotated Sorted Array II
- Find Min in Rotated Sorted Array
- Find Kth Rotation
- Single Element in a Sorted Array
- Find Peak Element
- Sqrt(x)
- Find nth root of m
- Koko Eating Bananas
- Minimum days to make M bouquets
- Min Divisor Search
- Ship Packages in D Days
- Kth Missing Positive Number
- Aggressive Cows
- Allocate Minimum Pages
- Split Array Largest Sum
- Median of Two Sorted Arrays
- Kth element of Two Sorted Arrays
- Row with max 1s
- Search a 2D Matrix
- Search a 2D Matrix II
- Find a Peak Element II
- Median in a row-wise sorted Matrix

Strings

- Remove Outermost Parentheses
- Reverse Words in a String
- Largest Odd Number in String
- Longest Common Prefix
- Isomorphic Strings
- Rotate String
- Valid Anagram
- Group Anagrams
- Roman to Integer

1752. Check if Array Is Sorted and Rotated

Array

Problem:

- Given an array nums, check if it was **sorted (non-decreasing)** and then **rotated** any number of times (including 0).
- Duplicates are allowed.
- Return true if valid, otherwise false.

Example:

Input: nums = [3,4,5,1,2], **Output:** true
Input: nums = [2,1,3,4], **Output:** false
Input: nums = [1,2,3], **Output:** true

Approach:

- Sort a copy of the array.
- Try **all possible rotations** of the sorted array.
- If any rotation equals the given array → true.

Complexity:

- **Time:** O(n^2) [Sorting: O(n log n) & Rotations: O(n^2)]
- **Space: Space:** O(n) — extra array

Dry Run

```java
class Solution {
    public boolean check(int[] nums) {
        int n = nums.length;
        // Step 1: Create sorted array
        int[] sorted = nums.clone();
        Arrays.sort(sorted);
        // Step 2: Try all rotations
        for (int shift = 0; shift < n; shift++) {
            boolean match = true;
            for (int i = 0; i < n; i++) {
                if (nums[i] != sorted[(i + shift) % n]) {
                    match = false;
                    break;
                }
            }
            if (match) return true;
        }
        return false;
    }
}
```

1752. Check if Array Is Sorted and Rotated

Array

Approach:

- A sorted array has **no decreasing pairs**.
- After rotation, there can be **at most one decrease point**.
- Count how many times nums[i] > nums[i+1] (circularly).
- More than one such case → not a valid rotation.

Complexity:

- **Time:** O(n) — single pass
- **Space:** O(1) — constant space

Takeaways:

- Rotated sorted array ⇒ **only one break in order**
- Circular traversal is a powerful pattern
- Great example of **observation-based optimization**

Dry Run

```java
class Solution {
    public boolean check(int[] nums) {
        int count = 0;
        int n = nums.length;

        for (int i = 0; i < n; i++) {
            if (nums[i] > nums[(i + 1) % n]) {
                count++;
            }
            if (count > 1) {
                return false;
            }
        }
        return true;
    }
}
```

26. Remove Duplicates from Sorted Array

Two Pointers | Array

Problem:

- Given a **sorted integer array nums**, remove duplicates **in-place**.
- Each unique element should appear **only once**, maintaining order.
- Return k = number of unique elements.
- First k positions of nums must contain the unique values.

Example:

Input: nums = [1,1,2]
Output: k = 2, nums = [1,2,_]

Input: nums = [0,0,1,1,1,2,2,3,3,4]
Output: k = 5, nums = [0,1,2,3,4,,,,,_]

Approach:

- Use **two pointers** to track unique elements.
- flag keeps the index of the **last unique element**.
- Traverse array once; if a new value is found, place it at flag + 1.
- Final unique count = flag + 1.

Complexity:

- **Time:** O(n) — single pass through the array
- **Space:** O(1) — in-place, no extra data structures

Dry Run

```java
class Solution {
    public int removeDuplicates(int[] nums) {
        int flag = 0;
        for(int i = 1; i < nums.length; i++){
            if(nums[flag] != nums[i]) {
                nums[flag + 1] = nums[i];
                flag++;
            }
        }
        return flag + 1;
    }
}
```

Takeaways:

- Sorted array → **duplicates are adjacent**
- Two-pointer technique is ideal for **in-place array modification**
- Common pattern for problems involving **unique elements in sorted data**

189. Rotate Array

Two Pointers | Array

Problem:

- Rotate array nums **to the right by k steps**.
- Rotation is **circular**.
- k may be larger than array length.

Example:

Input: [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]

Input: [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]

Approach:

- Store first n - k elements in a temp array.
- Move last k elements to the front.
- Append stored elements back.

Complexity:

- **Time:** O(n) — single pass through the array
- **Space:** O(n) — extra array ← Can be improved

Dry Run

```java
class Solution {
    public void rotate(int[] nums, int k) {
        int len = nums.length;
        int effRot = k % len;
        int[] tempArr = new int[len - effRot];

        for(int i = 0; i < len - effRot; i++) {
            tempArr[i] = nums[i];
        }

        int ind = 0;
        for(int i = len - effRot; i < len; i++) {
            nums[ind++] = nums[i];
        }

        for(int i = 0; i < len - effRot; i++) {
            nums[ind++] = tempArr[i];
        }

    }
}
```

#pattern

189. Rotate Array

Two Pointers | Array

Approach:

- Reduce rotations using k = k % n.
- Reverse entire array.
- Reverse first k elements.
- Reverse remaining n - k elements.

Complexity:

- **Time:** O(n) — single pass through the array
- **Space:** O(1) — in-place

Takeaways:

- Modulo avoids unnecessary rotations.
- Extra space solutions are easy but not optimal.
- **Reverse pattern** is key for in-place rotation problems.
- Always aim for **O(1) space** when modifying arrays.

Dry Run

```java
class Solution {
    public void rotate(int[] nums, int k) {
        k %= nums.length;
        int ind = nums.length - 1;
        reverse(nums, 0, ind);
        reverse(nums, 0, k - 1);
        reverse(nums, k, ind);
    }
    public void reverse(int[] arr, int start, int end) {
        while(start < end){
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
}
```

#pattern

283. Move Zeroes

Two Pointers | Array

Problem:

- Move all 0s to the **end of the array**.
- Maintain the **relative order of non-zero elements**.
- Must be done **in-place**.

Example:

Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]

Input: nums = [0]
Output: [0]

Takeaways:

- Two-pointer idea works well for **stable rearrangement**
- Useful pattern for **partitioning arrays in-place**
- Early exit avoids unnecessary processing when no zeros exist

Approach:

- First, find the **index of the first zero**.
- Traverse ahead; whenever a non-zero is found, **swap it with the zero index**.
- Increment zero index to keep pushing zeros to the right.
- Order of non-zero elements remains intact.

Complexity:

- **Time:** O(n) — single traversal
- **Space:** O(1) — in-place swaps

Dry Run

```java
class Solution {
    public void moveZeroes(int[] nums) {
        int flag = -1;
        int len = nums.length;
        for(int i = 0; i < len; i++) {
            if(nums[i] == 0) {
                flag = i;
                break;
            }
        }
        if(flag == -1) return;

        for(int i = flag + 1; i < len; i++) {
            if(nums[i] != 0) {
                swap(nums, flag, i);
                flag++;
            }
        }
    }

    public void swap(int[] arr, int a, int b) {
        int temp = arr[a];
        arr[a] = arr[b];
        arr[b] = temp;
    }
}
```

#pattern

268. Missing Number

Math | Array

Problem:

- Array nums contains **n distinct numbers** from range [0, n].
- Exactly **one number is missing**.
- Return the missing number.

Example:

Input: [0,1]
Output: 2

Input: [9,6,4,2,3,5,7,0,1]
Output: 8

Approach:

- Sort the array.
- Check index vs value mismatch.
- First mismatch index is the missing number.
- If none found, missing value is n.

Complexity:

- **Time:** O(n log n) — sorting
- **Space:** O(1) *(ignoring sort internals)*

Dry Run

```java
class Solution {
    public int missingNumber(int[] nums) {
        Arrays.sort(nums);
        for(int i = 0; i < nums.length; i++){
            if(i != nums[i]){
                return i;
            }
        }
        return nums.length;
    }
}
```

268. Missing Number

Math | Array

Approach:

- Sum of numbers from 0 to n is known.
- Subtract actual array sum from expected sum.
- Result gives the missing number.

Complexity:

- **Time:** O(n) – single
- **Space:** O(1)

Takeaways:

- Mathematical formulas can eliminate extra work.
- Sorting-based solutions are easy but not optimal.
- Try finding for **sum/math patterns** in range problems.

Dry Run

```java
class Solution {
    public int missingNumber(int[] nums) {
        int len = nums.length;
        int curSum = 0;
        int actSum = (len * (len + 1)) / 2;
        for(int i = 0; i < len; i++) {
            curSum += nums[i];
        }
        return actSum - curSum;
    }
}
```

485. Max Consecutive Ones

Array

Problem:

- Given a **binary array nums**, find the **maximum number of consecutive 1s**.
- Return the maximum streak length.

Example:

Input: [1,1,0,1,1,1]
Output: 3

Input: [1,0,1,1,0,1]
Output: 2

Approach:

- Traverse the array once.
- Maintain a counter for **current consecutive 1s**.
- Reset the counter when 0 is found.
- Track the **maximum count** seen so far.

Complexity:

- **Time:** O(n) — single traversal
- **Space:** O(1) — constant extra space

Dry Run

```java
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int cnt = 0;
        int isMax = 0;

        for(int i = 0; i < nums.length; i++) {
            if(nums[i] == 1) {
                cnt++;
            if(cnt > isMax) {
                isMax = cnt;
            }
            } else {
                cnt = 0;
            }
        }
        return isMax;
    }
}
```

Takeaways:

- Simple **running count** pattern.
- Reset-on-condition is a common array technique.
- Useful for **streak / subarray counting** problems.

136. Single Number

Math | Array

Problem:

- Array nums where **every element appears twice except one**.
- Find the **unique element**.

Example:

Input: [2,2,1]
Output: 1

Input: [4,1,2,1,2]
Output: 4

Input: [1]
Output: 1

Approach:

- Sort the array.
- Compare elements in pairs.
- First mismatch index is the missing number.
- If none, last element is unique.

Dry Run

```java
class Solution {
    public int singleNumber(int[] nums) {
        Arrays.sort(nums);
        int len = nums.length;
        for(int i = 0; i < len - 1; i = i + 2) {
            if(nums[i] != nums[i+1]){
                return nums[i];
            }
        }
        return nums[len - 1];
    }
}
```

Complexity:

- **Time:** O(n log n) – sorting
- **Space:** O(1) *(ignoring sort internals)*

136. Single Number

Math | Array

Approach:

- XOR all elements together.
- Duplicate numbers cancel out.
- Remaining value is the unique element.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Sorting works but violates optimal constraints.
- XOR is ideal for **pair + single element** problems.
- Always check for **bitwise tricks** in array problems.

Dry Run

```java
class Solution {
    public int singleNumber(int[] nums) {
        int xor = 0;
        for(int n : nums) {
            xor ^= n;
        }
        return xor;
    }
}
```

1. Two Sum

Hash Table | Array

Problem:

- Given array nums and integer target.
- Find **two distinct indices** whose values sum to target.
- Exactly **one valid solution** exists.

Example:

Input: nums = [2,7,11,15], target = 9
Output: [0,1]

Input: nums = [3,2,4], target = 6
Output: [1,2]

Input: nums = [3,3], target = 6
Output: [0,1]

Approach:

- Try all possible pairs.
- Check if their sum equals the target.
- Return indices when match is found.

Dry Run

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
    int [] ans = new int [2];

    for(int i = 0; i < nums.length - 1; i++) {
        int rem = target - nums[i];
        for(int j = i + 1; j < nums.length; j++){
            if(nums[j] == rem) {
                ans[0] = j;
                ans[1] = i;
                break;
            }
        }
    }
    return ans;
    }
}
```

Complexity:

- **Time:** $O(n^2)$ ← Can be improved
- **Space:** $O(1)$

#pattern

1. Two Sum

Hash Table | Array

Approach:

- Store visited numbers with their indices.
- For each element, check if its **complement** exists.
- Return indices immediately when found.

Complexity:

- **Time:** O(n)
- **Space:** O(n) — hash map

Takeaways:

- Complement-based thinking simplifies sum problems.
- Hashing trades **space for speed**.

Dry Run

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> numsHash = new HashMap<>();

        for(int i = 0; i < nums.length; i++) {
            int remVal = target - nums[i];

            if(numsHash.containsKey(remVal)){
                return new int[] {
                    numsHash.get(remVal), i
                };
            }

            numsHash.put(nums[i], i);
        }
        return new int[] {-1, -1};
    }
}
```

#pattern

75. Sort Colours

3 Pointers | Array

Problem:

- Array contains only 0, 1, 2 (red, white, blue).
- Sort the array **in-place** as 0 → 1 → 2.
- **No built-in sort** allowed.

Example:

Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]

Input: nums = [2,0,1]
Output: [0,1,2]

Approach:

- Count number of 0s, 1s, and 2s.
- Overwrite the array in order using counts.
- Simple and intuitive.

Complexity:

- **Time:** O(n) — two passes
- **Space:** O(1) — constant variables only

```java
class Solution {
    public void sortColors(int[] nums) {
        int zero = 0, one = 0, two = 0;

        for(int n: nums){
            if(n == 0) zero++;
            else if(n == 1) one++;
            else two++;
        }

        for(int i = 0; i < zero; i++) {
            nums[i] = 0;
        }

        for(int i = zero; i < zero + one; i++) {
            nums[i] = 1;
        }

        for(int i = zero + one; i < zero + one + two; i++) {
            nums[i] = 2;
        }
    }
}
```

#pattern

75. Sort Colours

3 Pointers | Array

#Dutch National Flag

Approach:

- Use three pointers: low, mid, high.
- Place 0 to the left, 2 to the right.
- Keep 1 in the middle automatically.
- Single pass, in-place.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Counting works but needs **two passes**.
- Dutch National Flag is **optimal & interview-favorite**.
- Three-pointer technique is powerful for **partition problems**.

#Pro Tip: mid is the decision pointer → it checks the current unknown element.

Dry Run

```java
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;

        while(mid <= high) {
            if(nums[mid] == 0) {
                int temp = nums[low];
                nums[low] = nums[mid];
                nums[mid] = temp;
                low++;
                mid++;
            } else if (nums[mid] == 1){
                mid++;
            } else {
                int temp = nums[mid];
                nums[mid] = nums[high];
                nums[high] = temp;
                high--;
            }
        }
    }
}
```

#pattern

169. Majority Element

Two Pointers | Array

Problem:

- Given array nums of size n.
- Find the **majority element** (appears more than ⌊n/2⌋ times).
- Majority element **always exists**.

Example:

Input: nums = [3,2,3]
Output: 3

Input: nums = [2,2,1,1,1,2,2]
Output: 2

Approach:

- Sort the array.
- Majority element must be at index n/2.
- Directly return middle element.

Complexity:

- **Time:** O(n log n) ← Can be improved
- **Space:** O(1) *(ignoring sort internals)*

```java
class Solution {
    public int majorityElement(int[] nums) {
        Arrays.sort(nums);
        return nums[nums.length/2];
    }
}
```

#pattern

169. Majority Element

Two Pointers | Array

#Boyer–Moore Voting

Approach:

- Maintain a candidate and a counter.
- Same value → increment count, different → decrement.
- When count becomes zero, pick new candidate.
- Final candidate is the majority element.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Sorting gives a quick but non-optimal solution.
- Boyer–Moore works because majority element **outnumbers all others combined**.
- Classic example of **counter-based greedy logic**.

Dry Run

```java
class Solution {
    public int majorityElement(int[] nums) {
        int cnt = 0;
        int cnd = 0;

        for(int num: nums) {
            if(cnt == 0){
                cnd = num;
            }

            if(num == cnd){
                cnt++;
            } else {
                cnt--;
            }
        }
        return cnd;
    }
}
```

#pattern

53. Maximum Subarray

Two Pointers | Array

Problem:

- Given an integer array nums.
- Find the **maximum sum of any contiguous subarray**.
- Return the maximum sum.

Example:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6

Input: nums = [5,4,-1,7,8]
Output: 23

Approach:

- Generate all possible subarrays.
- Calculate sum for each subarray.
- Track the maximum sum found.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Dry Run

```java
class Solution {
    public int maxSubArray(int[] nums) {
        int max = Integer.MIN_VALUE;
        for(int i = 0; i < nums.length; i++) {
         int sum = 0;
         for(int j = i; j < nums.length; j++) {
             sum += nums[j];
             max = Math.max(sum, max);
             }
        }
        return max;
    }
}
```

53. Maximum Subarray

Two Pointers | Array

#Kadane's Algorithm

Approach:

- Maintain a running subarray sum.
- Extend subarray if sum is positive.
- Reset sum when it becomes negative.
- Track the maximum sum during traversal.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Negative sums are harmful → reset early.
- Kadane's Algorithm is a **must-know** pattern.
- Applies to many **subarray optimization** problems.

Dry Run

```java
class Solution {
    public int maxSubArray(int[] nums) {
        int max = Integer.MIN_VALUE;
        int sum = 0;

        for(int i = 0; i < nums.length; i++) {
            sum += nums[i];

            if(sum > max) {
                max = sum;
            }

            if(sum < 0) {
                sum = 0;
            }
        }
        return max;
    }
}
```

121. Best Time to Buy and Sell Stock

Counting | Array

Problem:

- Given stock prices by day.
- Buy **once** and sell **later** to maximize profit.
- Return max profit; if none possible, return 0.

Example:

Input: prices = [7,1,5,3,6,4]
Output: 5

Input: prices = [7,6,4,3,1]
Output: 0

Approach:

- Try all buy–sell pairs.
- Compute profit for each pair.
- Track maximum profit.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Can be improved

```java
class Solution {
    public int maxProfit(int[] prices) {
        int len = prices.length;
        int max = 0;
        for (int i = 0; i < len; i++) {
            for (int j = i; j < len; j++) {
                max = Math.max(max, prices[j] - prices[i]);
            }
        }
        return max;
    }
}
```

#pattern

121. Best Time to Buy and Sell Stock

DP | Array

Approach:

- Track **minimum price so far**.
- Calculate profit if sold today.
- Update maximum profit.
- Always buy before selling.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Always track **best buy price** before selling.
- Single-pass greedy works for **one-transaction** problems.
- Foundation for advanced stock DP problems.

Dry Run

```java
class Solution {
    public int maxProfit(int[] prices) {
        int cost, profit = 0;
        int mini = prices[0];

        for(int i = 1; i < prices.length; i++) {
            cost = prices[i] - mini;
            profit = Math.max(cost, profit);
            mini = Math.min(mini, prices[i]);
        }
        return profit;
    }
}
```

#pattern

2149. Rearrange Array Elements by Sign

Array

Problem:

- Array nums has **equal positives and negatives**.
- Rearrange so **every adjacent pair has opposite signs**.
- Order of same-sign elements must be **preserved**.
- Result must **start with a positive** number.

Example:

Input: prices = [3,1,-2,-5,2,-4]
Output: [3,-2,1,-5,2,-4]

Approach:

- Separate positive and negative numbers into two lists.
- Place positives at **even indices** and negatives at **odd indices**.
- Fill alternately while preserving original order.

Complexity:

- **Time:** O(n) — single traversal + merge
- **Space:** O(n) — extra lists for positives & negatives

Takeaways:

- Stable order → extra space is acceptable.
- Index-based placement simplifies alternation.

```java
class Solution {
    public int[] rearrangeArray(int[] nums) {
        int n = nums.length;
        ArrayList<Integer> pos = new ArrayList<>();
        ArrayList<Integer> neg = new ArrayList<>();

        for(int i = 0; i < n; i++) {
            if(nums[i] > 0) {
            pos.add(nums[i]);
            } else {
                neg.add(nums[i]);
            }
        }

        for(int i = 0; i < n/2; i++) {
            nums[2 * i] = pos.get(i);
            nums[2 * i + 1] = neg.get(i);
        }
        return nums;
    }
}
```

31. Next Permutation

Two Pointers | Array

Problem:

- Given an array nums, generate the **next lexicographical permutation**.
- If no larger permutation exists, return the **smallest (sorted) order**.
- Must be done **in-place** using constant space.

Example:

Input: nums = [1,2,3] Output: [1,3,2]
Input: nums = [3,2,1] Output: [1,2,3]
Input: nums = [1,1,5] Output: [1,5,1]
Input: nums = [1,3,5,4,2] Output: [1,4,2,3,5]

Approach:

- Traverse from right to find **first decreasing element**.
- Find the **next larger element** to the right and swap.
- Reverse the suffix to get the **smallest greater arrangement**.
- If no decreasing point → array is last permutation → **sort ascending**.

Complexity:

- **Time:** O(n) — single scan + reverse
- **Space:** O(1) — in-place operations

Takeaways:

- Right-to-left traversal is key.
- Lexicographic problems follow a **patterned procedure**.

Dry Run

```java
class Solution {
    public void nextPermutation(int[] nums) {
        int ind = -1;
        int len = nums.length;
        //Step 1: Find the break point.
        for(int i = len - 2; i >= 0; i--) {
            if(nums[i] < nums[i+1]) {
                ind = i;
                break;
            }
        }
        // Step 2: Handle the edge case.
        if(ind == -1) {
            Arrays.sort(nums);
            return;
        }
        // Step 3: Find the successor to swap.
        for(int i = len - 1; i >= ind; i--) {
            if(nums[i] > nums[ind]) {
                swap(nums, i, ind);
                break;
            }
        }
        // Step 4: reversing it makes it sorted in ascending order
        // (lexicographically smallest).
        rev(nums, ind + 1, len - 1);
    }

    public void swap(int[] arr, int index1, int index2) {
        int temp = arr[index1];
        arr[index1] = arr[index2];
        arr[index2] = temp;
    }

    public void rev(int[] arr, int start, int end) {
        if(arr.length == 0) {
            return;
        }
        while (start < end) {
            swap(arr, start, end);
            start++;
            end--;
        }
    }
}
```

#pattern

Leaders in an Array

Two Pointers | Array

Problem:

- Find all **leader elements** in the array.
- An element is a **leader** if it is **greater than all elements to its right**.
- The **rightmost element** is always a leader.

Example:

Input: nums = [4,7,1,0] Output: [7, 1, 0]
Input: nums = [10,22,12,3,0,6] Output: [22, 12, 6]

Approach:

- Start traversal from the **right end**.
- Keep track of the **maximum seen so far**.
- If current element > max → it is a leader.
- Update max and store the leader.

Complexity:

- **Time:** O(n) – single reverse traversal
- **Space:** O(n) – to store leaders

Takeaways:

- Reverse traversal simplifies right-side comparison.
- Running maximum is a **powerful scanning pattern**.

Dry Run

```java
class Solution {
    public static ArrayList<Integer> findLeaders(int[] nums) {
        ArrayList<Integer> leads = new ArrayList<>();
        int len = nums.length;
        if (len == 0) {
            return leads;
        }
        int max = nums[len - 1];
        int cnt = 0;
        leads.add(nums[len - 1]);
        for(int i = len - 1; i >= 0; i--) {
            if(nums[i] > max) {
                cnt++;
                max = nums[i];
                leads.add(max);
            }
        }
        return leads;
    }
}
```

128. Longest Consecutive Sequence

Counting | Array

Problem:

- Given an **unsorted array**.
- Find the length of the **longest consecutive sequence**.
- Must run in **O(n) time**.

Example:

Input: nums = [100,4,200,1,3,2] Output: 4
Input: nums = [0,3,7,2,5,8,4,6,0,1] Output: 9
Input: nums = [1,0,1,2] Output: 3

Approach:

- Sort the array.
- Count consecutive increasing numbers.
- Reset count on breaks or duplicates.
- Track maximum length.

Complexity:

- **Time:** O(n log n)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int longestConsecutive(int[] nums) {
        if (nums.length == 0) {
            return 0;
        }
        Arrays.sort(nums);
        int cntCur = 0;
        int longest = 1;
        int lastSmallest = Integer.MIN_VALUE;
        for(int i = 0; i < nums.length; i++) {
            if(nums[i] - 1 == lastSmallest) {
                cntCur++;
                lastSmallest = nums[i];
            } else if(lastSmallest != nums[i]) {
                cntCur = 1;
                lastSmallest = nums[i];
            }
            longest = Math.max(cntCur, longest);
        }
        return longest;
    }
}
```

128. Longest Consecutive Sequence

Hash Table | Array

Approach: *inception_method :)*

- Store all elements in a set.
- Start counting only from numbers with **no previous neighbour**.
- Expand sequence forward.
- Track longest streak.

Complexity:

- **Time:** O(n)
- **Space:** O(n)

Takeaways:

- HashSet avoids unnecessary scanning.
- Start only from **sequence beginnings**.
- Classic problem for **set-based optimization**.

Dry Run

```java
class Solution {
    public int longestConsecutive(int[] nums) {
        if (nums.length == 0) {
            return 0;
        }
        int longest = 1;
        Set<Integer> st = new HashSet<>();
        for(int num : nums) {
            st.add(num);
        }
        for(int it : st) {
            if(!st.contains(it - 1)) {
                int cnt = 1;
                int curNum = it;
                while(st.contains(curNum + 1)) {
                    curNum++;
                    cnt++;
                }
                longest = Math.max(longest, cnt);
            }
        }
        return longest;
    }
}
```

#pattern

73. Set Matrix Zeroes

Matrix | Array

Problem:

- Given an m x n matrix.
- If any cell is 0, set its **entire row & column to 0**.
- Must be done **in-place**.

Example:

Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]

Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Approach:

- Use two arrays to mark **rows & columns** containing zero.
- Traverse matrix and mark positions.
- Second pass: set marked rows/columns to zero.

Complexity:

- **Time:** O(m × n)
- **Space:** O(m + n)

Dry Run

```java
class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        int[] row = new int [m];
        int[] col = new int [n];
        for(int i = 0; i < m; i++) {
            for(int j = 0; j < n; j++) {
                if(matrix[i][j] == 0) {
                    row[i] = 1;
                    col[j] = 1;
                }
            }
        }
        for(int i = 0; i < m; i++) {
            for(int j = 0; j < n; j++) {
                if(row[i] == 1 || col[j] == 1) {
                    matrix[i][j] = 0;
                }
            }
        }
    }
}
```

73. Set Matrix Zeroes

Matrix | Array

Approach:

- Use **first row & first column as markers**.
- Track first column separately.
- Zero out inner matrix using markers.
- Finally update first row & column if needed.

Complexity:

- **Time:** O(m × n)
- **Space:** O(1)

Takeaways:

- First row/column can act as **memory markers**.
- Reduces extra space to constant.
- Classic example of **space optimization**.

Dry Run

```java
class Solution {
    public void setZeroes(int[][] matrix) {
        int n = matrix.length;
        int m = matrix[0].length;
        // Use a variable to track if the first column needs to be zeroed
        int col0 = 1;
        // First pass: Use the first row and column as markers
        for(int i = 0 ; i < n ; i++){
            for(int j = 0 ; j < m ; j++){
                if(matrix[i][j] == 0){
                    // Mark the row
                    matrix[i][0] = 0;
                    // Mark the column. Handle the first column separately.
                    if(j != 0) {
                        matrix[0][j] = 0;
                    } else {
                        col0 = 0;
                    }
                }
            }
        }
        // Second pass: Use markers to set the inner matrix to zero
        for(int i = 1; i < n ; i++){
            for(int j = 1 ; j < m ; j++){
                if(matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
        // Final pass: Zero out the first row and/or column if needed
        if(matrix[0][0] == 0) {
            for(int j = 0; j < m; j++) {
                matrix[0][j] = 0;
            }
        }
        if(col0 == 0) {
            for(int i = 0; i < n; i++) {
                matrix[i][0] = 0;
            }
        }
    }
}
```

48. Rotate Image

Matrix | Array

Problem:

- Given an n x n matrix.
- Rotate the matrix **90° clockwise**.
- Must be done **in-place** (no extra matrix).

Example:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Approach:

- First **transpose** the matrix (swap matrix[i][j] with matrix[j][i]).
- Then **reverse each row**.

Complexity:

- **Time:** $O(n^2)$
- **Space:** O(1) — in-place

Takeaways:

- Rotation = **Transpose + Reverse rows**. No extra space needed.

Dry Run

```java
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        // Step 1: Transpose Matrix
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        // Step 2: Reverse each row
        // We iterate through every row (i)
        for (int i = 0; i < n; i++) {
            // Standard two-pointer reversal
            int left = 0;
            int right = n - 1;
            while (left < right) {
                int temp = matrix[i][left];
                matrix[i][left] = matrix[i][right];
                matrix[i][right] = temp;
                left++;
                right--;
            }
        }
    }
}
```

54. Spiral Matrix

Matrix | Array

Problem:

- Given an m x n matrix.
- Return all elements in **spiral order** (clockwise).
- Start from **top-left**.

Example:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Approach:

- Maintain **four boundaries**: top, bottom, left, right.
- Traverse:
  - Left → Right (top row)
  - Top → Bottom (right column)
  - Right → Left (bottom row)
  - Bottom → Top (left column)
- Shrink boundaries after each pass.
- Continue until all elements are visited.

Complexity:

- **Time:** O(m × n)
- **Space:** O(1)

Takeaways:

- Boundary-based traversal is key.
- Careful with **overlapping conditions**.

Dry Run

```java
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> result = new ArrayList<>();
        if (matrix == null || matrix.length == 0) {
            return result;
        }
        int top = 0;
        int bottom = matrix.length - 1;
        int left = 0;
        int right = matrix[0].length - 1;
        while (top <= bottom && left <= right) {
            for (int j = left; j <= right; j++) {
                result.add(matrix[top][j]);
            }
            top++;
            for (int i = top; i <= bottom; i++) {
                result.add(matrix[i][right]);
            }
            right--;
            if (top <= bottom) {
                for (int j = right; j >= left; j--) {
                    result.add(matrix[bottom][j]);
                }
                bottom--;
            }
            if (left <= right) {
                for (int i = bottom; i >= top; i--) {
                    result.add(matrix[i][left]);
                }
                left++;
            }
        }
        return result;
    }
}
```

560. Subarray Sum Equals K

Counting | Array

Problem:

- Given an integer array nums and integer k.
- Find the **count of subarrays** whose sum equals k.
- Subarray must be **contiguous**.

Example:

Input: nums = [1,1,1], k = 2
Output: 2

Input: nums = [1,2,3], k = 3
Output: 2

Approach:

- Generate all possible subarrays.
- Calculate sum for each subarray.
- Increment count when sum equals k.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Dry Run

```java
class Solution {
    public int subarraySum(int[] nums, int k) {
        int n = nums.length;
        int cnt = 0;
        for(int i = 0; i < n; i++) {
            int sum = 0;
            for(int j = i; j < n; j++) {
                sum += nums[j];
                if(sum == k) cnt++;
            }
        }
        return cnt;
    }
}
```

#pattern

560. Subarray Sum Equals K

Prefix Sum | Array

Approach:

- Maintain running prefix sum.
- Check if (currentSum - k) exists in map.
- Add frequency to count.
- Store prefix sum occurrences.

This soln came from a qns i.e.
*Why **Integer::sum***
*Why not **Sunny::xyz***

Complexity:

- **Time:** O(n)
- **Space:** O(n) — HashMap

Java | Auto

```java
class Solution {
    public int subarraySum(int[] nums, int k) {
        int cnt = 0, prefixSum = 0;
        HashMap<Integer, Integer> prefixSumCnt = new HashMap<>();
        prefixSumCnt.put(0,1);

        for(int i = 0; i < nums.length; i++) {
            prefixSum += nums[i];
            int rem = prefixSum - k;
            if(prefixSumCnt.containsKey(rem)) {
                cnt += prefixSumCnt.get(rem);
            }
            prefixSumCnt.merge(prefixSum, 1, Solution :: xyz);
        }
        return cnt;
    }

    public static int xyz(int oldVal, int newVal) {
    return oldVal + newVal;
    }
}
```

```java
class Solution {
    public int subarraySum(int[] nums, int k) {
        int n = nums.length;
        int cnt = 0;
        int prefixSum = 0;
        HashMap<Integer, Integer> prefixSumCount = new HashMap<>();
        prefixSumCount.put(0, 1);
        for(int i = 0; i < n; i++) {
            prefixSum += nums[i];
            int remove = prefixSum - k;

            if(prefixSumCount.containsKey(remove)) {
             cnt += prefixSumCount.get(remove);
            }
        prefixSumCount.put(prefixSum, prefixSumCount.getOrDefault(prefixSum, 0) + 1);
        }
        return cnt;
    }
}
```

Why this, Think!
Do a DRY run for
{3, -3, 1 ,1, 1} :: k = 3

Bonus Tip: A method reference like Solution::xyz is just a shortcut for a lambda expression *(oldVal, newVal) -> oldVal + newVal*, which Java uses to pass actual logic as a method argument.

Takeaways:

- Hashing helps count efficiently.
- Key pattern for **range sum problems**.
- Prefix sum converts subarray sum to **difference problem**.

NOTE

prefixSumCount.put(prefixSum, prefixSumCount.getOrDefault(prefixSum, 0) + 1);
Can be replaced with ***prefixSumCount.merge(prefixSum, 1, Integer::sum);***

#pattern

229. Majority Element II

Hash Table | Array

Problem:

- Given an array nums, find all elements that appear **more than ⌊n/3⌋ times**.
- At most **2 such elements** can exist.
- Return them in a list.

Example:

Input: nums = [3,2,3] Output: [3]
Input: nums = [1] Output: [1]
Input: nums = [1,2] Output: [1,2]

Approach:

- Count frequency of each number using a map.
- Add element to result when its count reaches ⌊n/3⌋ + 1.

Complexity:

- **Time:** O(n)
- **Space:** O(n) – HashMap

Dry Run

```java
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        ArrayList<Integer> ans = new ArrayList<>();
        int n = nums.length;
        int target = (int)(n / 3) + 1;

        HashMap<Integer, Integer> hash = new HashMap<>();
        for(int i = 0; i < n; i++) {
           hash.put(nums[i], hash.getOrDefault(nums[i], 0) + 1);
            if(hash.get(nums[i]) == target) {
                ans.add(nums[i]);
            }
        }
        return ans;
    }
}
```

229. Majority Element II

Counting | Array

#Boyer–Moore Voting

Approach:

- Track **two potential candidates** and their counts.
- First pass: find possible candidates.
- Second pass: verify their actual frequency.

Complexity:

- **Time:** O(n)
- **Space:** O(1) – constant space

Takeaways:

- More than n/3 ⇒ **maximum 2 majority elements**
- HashMap is easy but **space-heavy**
- Boyer-Moore is **interview favorite** for majority problems
- Pattern extends to n/k type questions

```java
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int cnt1 = 0;
        int cnt2 = 0;
        int ele1 = Integer.MIN_VALUE;
        int ele2 = Integer.MIN_VALUE;
        //Getting the two elements with max occurrence
        for(int n : nums) {
            if(cnt1 == 0 && ele2 != n) {
                cnt1 = 1;
                ele1 = n;
            } else if (cnt2 == 0 && ele1 != n) {
                cnt2 = 1;
                ele2 = n;
            } else if (n == ele1) cnt1++;
              else if (n == ele2) cnt2++;
              else {
                cnt1--;
                cnt2--;
              }
        }
        int c1 = 0;
        int c2 = 0;
        List<Integer> ans = new ArrayList<>();
        //Checking if it's more than ⌊ n/3 ⌋ times
        for(int n : nums){
            if(ele1 == n) c1++;
            if(ele2 == n) c2++;
        }
        int target = (int)(nums.length / 3) + 1;
        if(c1>= target) ans.add(ele1);
        if(c2>= target) ans.add(ele2);
        return ans;
    }
}
```

15. 3 Sum

Hash Table | Array

Problem:

- Find all **unique triplets** [a,b,c] such that a + b + c = 0
- Indices must be different.
- Result must **not contain duplicate triplets**.

Example:

Input: nums = [-1,0,1,2,-1,-4] Output: [[-1,-1,2],[-1,0,1]]
Input: nums = [0,1,1] Output: []
Input: nums = [0,0,0] Output: [[0,0,0]]

Approach:

- Fix one element, use HashSet for 2-sum.
- Check if third value exists.
- Sort triplet and store in Set to avoid duplicates.

Complexity:

- **Time:** O(n^2 log k) *(sorting triplets)*
- **Space:** O(n) – HashSet

Dry Run

```java
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        int n = nums.length;
        HashSet<List<Integer>> triplets = new HashSet<>();
        for(int i = 0; i < n; i++) {
            HashSet<Integer> hashVal = new HashSet<>();
            for(int j = i + 1; j < n; j++) {
                int thirdVal = -(nums[i] + nums[j]);
                if(hashVal.contains(thirdVal)) {
                   List<Integer> triplet = new ArrayList<>();
                    triplet.add(nums[i]);
                    triplet.add(nums[j]);
                    triplet.add(thirdVal);
                    Collections.sort(triplet);
                    triplets.add(triplet);
                }
                hashVal.add(nums[j]);
            }
        }
       List<List<Integer>> ans = new ArrayList(triplets);
       return ans;
    }
}
```

#pattern

15. 3 Sum

Two Pointer | Array

Approach:

- Sort the array first.
- Fix one element, use two pointers for remaining sum.
- Skip duplicates to maintain uniqueness.
- Move pointers based on sum.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$ *(excluding result list)*

Takeaways:

- Sorting simplifies **duplicate handling**
- Two-pointer is best for **pair sum problems**
- HashSet approach is intuitive but **not optimal**
- Classic example of **reduce 3Sum → 2Sum**

Dry Run

```java
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> ans = new ArrayList<>();
        Arrays.sort(nums);
        int n = nums.length;
        for(int i = 0; i < n; i++) {
            if(i != 0 && nums[i] == nums[i - 1]) continue;
            int l = i + 1;
            int r = n - 1;
            while(l < r) {
                int sum = nums[i] + nums[l] + nums[r];
                if(sum > 0) r--;
                else if(sum < 0) l++;
                else {
                    List<Integer> triplet = Arrays.asList(nums[i], nums[l], nums[r]);
                    ans.add(triplet);
                    l++;
                    r--;
                    while(l < r && nums[l] == nums[l - 1]) l++;
                    while(l < r && nums[r] == nums[r + 1]) r--;
                }
            }
        }
        return ans;
    }
}
```

#pattern

18. 4Sum

Hash Table | Array

Problem:

- Find all **unique quadruplets** [a,b,c,d] such that a + b + c + d = target
- All indices must be **distinct**.
- Output can be in **any order**.

Example:

Input: nums = 1,0,-1,0,-2,2] k = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]

Approach:

- Fix **two numbers**.
- Use HashSet to find remaining two values.
- Sort quadruplet & store in Set to avoid duplicates.

Complexity:

- **Time:** $O(n^3 \log k)$
- **Space:** O(n) – HashSet

Dry Run

```java
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        Set<List<Integer>> quadruplets = new HashSet<>();
        int n = nums.length;
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                HashSet<Integer> hashVal = new HashSet<>();
                for(int k = j + 1; k < n; k++) {
                    long required = (long) target - nums[i] - nums[j] - nums[k];
                     if (hashVal.contains((int) required)) {
                        List<Integer> temp =
                        Arrays.asList(nums[i], nums[j], nums[k], (int) required);
                        Collections.sort(temp);
                        quadruplets.add(temp);
                    }
                    hashVal.add(nums[k]);
                }
            }
        }
        return new ArrayList<>(quadruplets);
    }
}
```

18. 4Sum

Two Pointer | Array

Approach:

- Sort the array first.
- Fix **two numbers**, apply two pointers for rest.
- Skip duplicates at every step.
- Adjust pointers based on sum.

Complexity:

- **Time:** $O(n^3)$
- **Space:** $O(1)$ *(excluding result list)*

Takeaways:

- 4Sum is an extension of **3Sum / 2Sum**
- Sorting simplifies **duplicate handling**
- Two-pointer reduces brute force complexity
- Use long to avoid **overflow**

Dry Run

```java
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> ans = new ArrayList<>();
        int n = nums.length;
         Arrays.sort(nums);
         for(int i = 0; i < n; i++) {
             if(i > 0 && nums[i] == nums[i - 1]) continue;
             for(int j = i + 1; j < n; j++) {
                 if(j > i + 1 && nums[j] == nums[j - 1]) continue;
                 int l = j + 1;
                 int r = n - 1;
                 while(l < r) {
                     long sum = (long) nums[i] + nums[j] + nums[l] + nums[r];
                     if(sum == (long) target) {
                         ans.add(Arrays.asList(nums[i], nums[j], nums[l], nums[r]));
                     while(l < r && nums[l] == nums[l + 1]) l++;
                     while(l < r && nums[r] == nums[r - 1]) r--;
                     l++;
                     r--;
                     }
                     else if (sum > target) r--;
                     else l++;
                 }
             }
         }
         return ans;
    }
}
```

Largest subarray with 0 sum

Prefix Sum | Array

Problem:

- Given an array with **positive & negative numbers**.
- Find the **length of the longest subarray** whose sum is 0.

Example:

Input: nums = [15,-2,2,-8,1,7,10,23] Output: 5
Input: nums = [1,0,-4,3,1,0] Output: 5
Input: nums = [2,10,4] Output: 0

Approach:

- Maintain **running prefix sum**.
- Store first occurrence of each sum in HashMap.
- If same sum appears again → subarray sum = 0.
- Track **maximum length** using index difference.
- Special case: if prefix sum becomes 0.

Dry Run

```java
class Solution {
    int maxLength(int arr[]) {
        HashMap<Integer, Integer> mp = new HashMap<Integer, Integer>();
        int maxLen = 0;
        int sum = 0;
        for(int i = 0; i < arr.length; i++) {
            sum += arr[i];
            if(sum == 0) {
                maxLen = i + 1;
            } else {
                if(mp.containsKey(sum)) {
                    maxLen = Math.max(maxLen,  i - mp.get(sum));
                }
                else {
                    mp.put(sum, i);
                }
            }
        }
        return maxLen;
    }
}
```

Complexity:

- **Time:** O(n) — single pass
- **Space:** O(n) — HashMap

Takeaways:

- Prefix sum helps detect **zero-sum subarrays**
- Store **first occurrence only** for max length
- Works with **negative numbers** (sliding window fails)
- Very common **HashMap pattern**

Count Subarrays with given XOR

Counting | Array

Problem:

- Given an integer array arr and a value k.
- Count **number of subarrays** whose XOR equals k.
- Subarray must be **contiguous**.

Example:

Input: nums = [4,2,2,6,4], k = 6 Output: 4
Input: nums = [5,6,7,8,9], k = 5 Output: 2
Input: nums = [1,1,1,1], k = 0 Output: 4

Approach:

- Generate **all subarrays**.
- Compute XOR for each.
- Increment count when XOR == k.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Dry Run

```java
class Solution {
    public long subarrayXor(int arr[], int k) {
        int cnt = 0;
        int n = arr.length;
        for(int i = 0; i < n; i++) {
            int xor = 0;
            for(int j = i; j < n; j++) {
                xor ^= arr[j];
                if(xor == k) cnt++;
            }
        }
        return cnt;
    }
}
```

Count Subarrays with given XOR

Prefix Sum | Array

Approach:

- Maintain **running prefix XOR**.
- If prefixXor ^ k exists → valid subarray found.
- Store frequency of prefix XORs.
- Accumulate count using HashMap.

Complexity:

- **Time:** O(n)
- **Space:** O(n) – HashMap

Takeaways:

- XOR problems follow **prefix pattern**
- prefixXor ^ k trick is key
- HashMap converts **O(n²) → O(n)**
- Similar to **subarray sum** logic

Dry Run

```java
class Solution {
    public long subarrayXor(int arr[], int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        freq.put(0, 1);
        int prefixXor = 0;
        int count = 0;
        for (int num : arr) {
            prefixXor ^= num;
            int target = prefixXor ^ k;
            if (freq.containsKey(target)) {
                count += freq.get(target);
            }
            freq.put(prefixXor, freq.getOrDefault(prefixXor, 0) + 1);
        }
        return count;
    }
}
```

56. Merge Intervals

Sorting | Array

Problem:

- Given a list of intervals [start, end].
- Merge all **overlapping intervals**.
- Return **non-overlapping intervals** covering all ranges.

Example:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]

Approach:

- **Sort intervals** by start time.
- Traverse intervals one by one.
- If current interval **overlaps** with last merged:
  - Merge by extending end time.
  - Else, Add as a **new interval**.

Complexity:

- **Time:** O(n log n) – sorting
- **Space:** O(n) – result list

Dry Run

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        int n = intervals.length;
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> ans = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if (ans.isEmpty() || intervals[i][0] > ans.get(ans.size() - 1)[1]) {
                ans.add(intervals[i]);
            } else {
                ans.get(ans.size() - 1)[1] =
                Math.max(ans.get(ans.size() - 1)[1], intervals[i][1]);
            }
        }
        return ans.toArray(new int[ans.size()][]);
    }
}
```

Takeaways:

- Sorting simplifies **overlap detection**
- Always compare with **last merged interval**
- Classic **interval merging pattern**
- Useful in scheduling & range problems

88. Merge Sorted Array

Sorting | Array

Problem:

- Given two **sorted arrays** nums1 and nums2.
- Merge them into **nums1** in sorted order.
- nums1 has extra space at the end to hold elements of nums2 i.e. ***nums1.length == m + n && nums2.length == n***

Example:

Input: nums1 = [1,2,3,0,0,0], m = 3
nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]

Input: nums1 = [1], m = 1, nums2 = [], n = 0
Output: [1]

Approach:

- Copy all elements of nums2 into empty positions of nums1.
- Sort the entire nums1 array.

```java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = 0;
        while(m + i < m + n) {
            nums1[m + i] = nums2[i];
            i++;
        }
        Arrays.sort(nums1);
    }
}
```

Complexity:

- **Time:** O((m+n) log(m+n))
- **Space:** O(1)

88. Merge Sorted Array

Three Pointers | Array

Approach:

- Use three pointers from **end of arrays**.
- Compare largest elements and place at correct position.
- Fill from back to avoid overwriting valid data.

Complexity:

- **Time:** O(m+n)
- **Space:** O(1) - in-place

Takeaways:

- Always merge **from the end** when space is available.
- Avoid sorting when arrays are already sorted.
- Two-pointer technique is **most efficient** here.
- Classic **in-place merge pattern**

Pro tip:

- Use m and n to initialize pointers, **not array lengths**.
- nums1 has extra buffer space (0s) that should be ignored during comparisons.

Dry Run

```java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int p1 = m - 1;
        int p2 = n - 1;
        int p = m + n - 1;
        while (p1 >= 0 && p2 >= 0) {
            if (nums1[p1] > nums2[p2]) {
                nums1[p] = nums1[p1];
                p1--;
            } else {
                nums1[p] = nums2[p2];
                p2--;
            }
            p--;
        }
        while (p2 >= 0) {
            nums1[p] = nums2[p2];
            p2--;
            p--;
        }
    }
}
```

Merge Without Extra Space

Sorting | Array

Problem:

- Given two **sorted arrays** a[] and b[].
- Merge them **without using extra space**.
- After merge:
  - a[] contains first n elements
  - b[] contains last m elements

Example:

Input: a = [2,4,7,10], b = [2,3]
Output: a = [2,2,3,4], b = [7,10]

Input: a = [1,5,9,10,15,20], b = [2,3,8,13]
Output: a = [1,2,3,5,8,9], b = [10,13,15,20]

Approach:

- Treat both arrays as **one combined array**.
- Use a **gap** to compare elements far apart.
- Swap if elements are out of order.
- Reduce gap gradually (like Shell Sort).
- Continue until gap becomes 1.

Dry Run

```java
class Solution {
    public void mergeArrays(int a[], int b[]) {
        int len = a.length + b.length;
        int gap = (len / 2) + (len % 2);
        while (gap > 0) {
            int left = 0;
            int right = left + gap;
            while (right < len) {
                if (left < a.length && right >= a.length) {
                    swapIfGreater(a, b, left, right - a.length);
                }
                else if (left >= a.length) {
                    swapIfGreater(b, b, left - a.length, right - a.length);
                }
                else {
                    swapIfGreater(a, a, left, right);
                }
                left++;
                right++;
            }
            if (gap == 1) break;
            gap = (gap / 2) + (gap % 2);
        }
    }
        private void swapIfGreater(int[] nums1, int[] nums2, int ind1, int ind2) {
        if (nums1[ind1] > nums2[ind2]) {
            int temp = nums1[ind1];
            nums1[ind1] = nums2[ind2];
            nums2[ind2] = temp;
        }
    }
}
```

Merge Without Extra Space

Sorting | Array

Complexity:

- **Time:** O((n+m) log(n+m))
- **Space:** O(1) — in-place

Takeaways:

- Gap method avoids **extra space**
- Inspired by **Shell Sort**
- Useful for **in-place merging problems**
- Handles cross-array swapping efficiently

Dry Run

```java
class Solution {
    public void mergeArrays(int a[], int b[]) {
        int len = a.length + b.length;
        int gap = (len / 2) + (len % 2);
        while (gap > 0) {
            int left = 0;
            int right = left + gap;
            while (right < len) {
                if (left < a.length && right >= a.length) {
                    swapIfGreater(a, b, left, right - a.length);
                }
                else if (left >= a.length) {
                    swapIfGreater(b, b, left - a.length, right - a.length);
                }
                else {
                    swapIfGreater(a, a, left, right);
                }
                left++;
                right++;
            }
            if (gap == 1) break;
            gap = (gap / 2) + (gap % 2);
        }
    }
        private void swapIfGreater(int[] nums1, int[] nums2, int ind1, int ind2) {
        if (nums1[ind1] > nums2[ind2]) {
            int temp = nums1[ind1];
            nums1[ind1] = nums2[ind2];
            nums2[ind2] = temp;
        }
    }
}
```

Count Inversions

Sorting | Array

Problem:

- Given an integer array arr.
- Count number of **inversion pairs (i, j)** such that:
  - i < j
  - arr[i] > arr[j]

Example:

Input: arr = [2,4,1,3,5] Output: 3
Input: arr = [10,10,10] Output: 0
Input: arr = [2,3,4,5,6] Output: 0

Approach:

- Check **every possible pair (i, j)**.
- If i < j and arr[i] > arr[j], increment count.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

#If I am allowed to rearrange the order of the given array then I can come up with an more optimized solution!

```java
class Solution {
    static int inversionCount(int arr[]) {
        int cnt = 0;
        int n = arr.length;
        for(int i = 0; i < n; i++) {
            for(int j = i = 1; j < n; j++) {
                if(arr[i] > arr[j]) cnt++;
            }
        }
        return cnt;
    }
}
```

Count Inversions

Sorting | Array

Approach:

- Use **divide & conquer**.
- Count inversions in:
  - Left half
  - Right half
  - During merge step
- While merging:
  - If left element > right element
  - all remaining left elements form inversions.

Complexity:

- **Time:** O(n log n)
- **Space:** O(n) – temp array

Takeaways:

- Brute force is simple but **slow**
- Merge Sort efficiently counts **split inversions**
- Classic example of **divide & conquer**
- Important problem for **array analysis**

Dry Run

```java
class Solution {
    static int inversionCount(int arr[]) {
        if (arr == null || arr.length < 2) {
            return 0;
        }
        return mergeSortAndCount(arr, 0, arr.length - 1);
    }
    private static int mergeSortAndCount(int[] arr, int low, int high) {
        int cnt = 0;
        if (low >= high) {
            return 0;
        }
        int mid = low + (high - low) / 2;
        cnt += mergeSortAndCount(arr, low, mid);
        cnt += mergeSortAndCount(arr, mid + 1, high);
        cnt += merge(arr, low, mid, high);
        return cnt;
    }
 private static int merge(int[] arr, int low, int mid, int high) {
        int[] temp = new int[high - low + 1];
        int left = low;
        int right = mid + 1;
        int k = 0;
        int cnt = 0;
        while (left <= mid && right <= high) {
            if (arr[left] <= arr[right]) {
                temp[k++] = arr[left++];
            } else {
                temp[k++] = arr[right++];
                cnt += (mid - left + 1);
            }
        }
        while (left <= mid) {
            temp[k++] = arr[left++];
        }
        while (right <= high) {
            temp[k++] = arr[right++];
        }
        for (int i = 0; i < temp.length; i++) {
            arr[low + i] = temp[i];
        }
        return cnt;
    }
}
```

493. Reverse Pairs

Sorting | Array

Problem:

- Given an integer array nums.
- Count pairs (i, j) such that:
  - i < j
  - nums[i] > 2 * nums[j]

Example:

Input: nums = [1,3,2,3,1] Output: 2
Input: nums = [2,4,3,5,1] Output: 3

Approach:

- Check **all possible pair (i, j)**.
- If condition nums[i] > 2 * nums[j] holds → count it.

Complexity:

- **Time:** O(n^2)
- **Space:** O(1)

```java
class Solution {
    public int reversePairs(int[] nums) {
        int cnt = 0;
        for(int i = 0; i < nums.length; i++) {
            for(int j = i + 1; j < nums.length; j++) {
                if(nums[i] > 2 * (long)nums[j]) cnt++;
            }
        }
        return cnt;
    }
}
```

#If I am allowed to rearrange the order of the given array then I can come up with an more optimized solution!

493. Reverse Pairs

Sorting | Array

Approach:

- Use **divide & conquer**.
- Recursively sort left & right halves.
- Before merging:
  - Count valid pairs using two pointers.
- Merge both halves in sorted order.

Complexity:

- **Time:** O(n log n)
- **Space:** O(n) – temp array

Takeaways:

- Similar to **Inversion Count** problem
- Use long to avoid **overflow**
- Two-pointer counting before merge is key
- Classic **merge sort optimization**

Dry Run

```java
class Solution {
    public int reversePairs(int[] nums) {
        if (nums == null || nums.length == 0) return 0;
        return mergeSort(nums, 0, nums.length - 1);
    }
    private int mergeSort(int[] arr, int low, int high) {
        int cnt = 0;
        if (low >= high) return cnt;
        int mid = low + (high - low) / 2;
        cnt += mergeSort(arr, low, mid);
        cnt += mergeSort(arr, mid + 1, high);
        cnt += countPairs(arr, low, mid, high);
        merge(arr, low, mid, high);
        return cnt;
    }
    private int countPairs(int[] arr, int low, int mid, int high) {
        int right = mid + 1;
        int cnt = 0;
        for (int i = low; i <= mid; i++) {
        while (right <= high && (long) arr[i] > 2L * arr[right]) {
                right++;
            }
            cnt += (right - (mid + 1));
        }
        return cnt;
    }
     private void merge(int[] arr, int low, int mid, int high) {
        ArrayList<Integer> temp = new ArrayList<>();
        int left = low;
        int right = mid + 1;
        while (left <= mid && right <= high) {
            if (arr[left] <= arr[right]) {
                temp.add(arr[left++]);
            } else {
                temp.add(arr[right++]);
            }
        }
        while (left <= mid) {
            temp.add(arr[left++]);
        }
        while (right <= high) {
            temp.add(arr[right++]);
        }
        for (int i = 0; i < temp.size(); i++) {
            arr[low + i] = temp.get(i);
        }
    }
}
```

152. Maximum Product Subarray

Counting | Array

Problem:

- Given an integer array nums.
- Find a **contiguous subarray** with the **maximum product**.
- Return that maximum product.

Example:

Input: nums = [2,3,-2,4]
Output: 6

Input: nums = [-2,0,-1]
Output: 0

Approach:

- Generate **all subarrays**.
- Calculate product for each.
- Track the maximum product.

Complexity:

- **Time:** $O(n^2)$
- **Space:** $O(1)$

Dry Run

```java
class Solution {
    public int maxProduct(int[] nums) {
        int max = Integer.MIN_VALUE;
        int n = nums.length;
        for(int i = 0; i < n; i++) {
            int prod = 1;
            for(int j = i; j < n; j++) {
                prod *= nums[j];
                max = Math.max(max, prod);
            }
        }
        return max;
    }
}
```

152. Maximum Product Subarray

DP | Array

Approach:

- Traverse from **left to right** (prefix product).
- Traverse from **right to left** (suffix product).
- Reset product when 0 appears.
- Track maximum from both directions.
- Handles **negative number sign flips**.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Takeaways:

- Negative numbers can **flip product sign**
- Zero breaks subarray → **reset product**
- Prefix + suffix ensures **no missed cases**
- Observation-based problems save heavy computation

Dry Run

```java
class Solution {
    public int maxProduct(int[] nums) {
        int n = nums.length;
        int ans = Integer.MIN_VALUE;
        int pre = 1;
        int suf = 1;
        for(int i = 0; i < n; i++) {
            if(pre == 0) pre = 1;
            if(suf == 0) suf = 1;
            pre *= nums[i];
            suf *= nums[n - i - 1];
            ans = Math.max(ans, Math.max(suf, pre));
        }
        return ans;
    }
}
```

1838. Frequency of the Most Frequent Element

Two Pointer | Array

Problem:

- Given array nums and k operations.
- In one operation, increment any element by 1.
- Find the **maximum possible frequency** of any element after at most k operations.

Example:

Input: nums = [1,2,4], k = 5
Output: 3

Input: nums = [1,4,8,13], k = 5
Output: 2

Approach:

- Sort the array to bring similar values together.
- Treat the rightmost element as the **target value**.
- Use a sliding window to check if all elements in the window can be increased to the target using at most k operations.
- Shrink the window when required operations exceed k.
- Track the maximum valid window size.

Complexity:

- **Time:** O(n log n) — sorting + sliding window
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int maxFrequency(int[] nums, int k) {
        Arrays.sort(nums);
        long total = 0, maxFreq = 0;
        int l = 0, r = 0, n = nums.length;

        while(r < n) {
            total += nums[r];

            while(nums[r] * (r - l + 1L) > total + k) {
                total -= nums[l];
                l++;
            }
            maxFreq = Math.max(maxFreq, r - l + 1L);
            r++;
        }
        return (int) maxFreq;
    }
}
```

Takeaways:

- Sorting helps convert the problem into a **window expansion** problem.
- Sliding window is useful when maintaining a **valid range under constraints**.

Longest subarray with given sum K (non-negative)

Two Pointer | Array

Problem:

- Given an array nums containing **only positive integers** and an integer k.
- Find the length of the **longest subarray whose sum equals k**.

Example:

Input: nums = [1,2,3,1,1,1,1], k = 6 Output: 4

Approach:

- Use two pointers (left and right) to maintain a sliding window.
- Expand the window by moving right and adding elements to the sum.
- If the sum exceeds k, shrink the window from the left.
- Whenever the sum becomes exactly k, update the maximum length.
- Since all numbers are positive, removing elements always decreases the sum, making the sliding window valid.

Dry Run

```java
public static int longestSubarrWithSumKInNonNegArr(int[] nums, int k) {
    int left = 0, right = 0;
    long sum = nums[0];
    int maxLen = 0;
    int n = nums.length;
    while(right < n) {
    while (left <= right && sum > k){
        sum -= nums[left];
        left++;
    }
    if(sum == k) {
        maxLen = Math.max(maxLen, right - left + 1);
    }
        right++;
        if(right < n) sum += nums[right];
    }
return maxLen;
}
```

Complexity:

- **Time:** O(n) — each pointer moves at most n times
- **Space:** O(1)

Takeaways:

- Sliding Window works when the array contains only positive numbers.
- Positive numbers guarantee predictable sum behavior during expansion and shrinking.

Longest subarray with given sum K

Hash Map | Array

Problem:

- Given an array nums (may contain positive, negative, and zero values) and an integer k.
- Find the length of the **longest subarray whose sum equals k**.

Example:

Input: nums = [1,-1,5,-2,3], k = 3 Output: 4

Approach:

- Maintain a running prefix sum while traversing the array.
- Let sum be the prefix sum till the current index.
- If sum == k, the subarray from index 0 to i is valid.
- Compute rem = sum - k.
- If rem has been seen before, then the subarray between the previous index and the current index sums to k.
- Store the **first occurrence** of every prefix sum in a HashMap to maximize the subarray length.

Dry Run

```java
public static int longestSubarrWithSumK(int[] nums, long k) {
    HashMap<Long, Integer> preSumMap = new HashMap<>();
    long sum = 0;
    int maxLen = 0;
    for (int i = 0; i < nums.length; i++) {
        sum += nums[i];
        // If the entire subarray from 0 to i has sum k
        if (sum == k) {
            maxLen = Math.max(maxLen, i + 1);
        }
        long rem = sum - k;
        // Check if there exists a prefix sum = sum - k
        if (preSumMap.containsKey(rem)) {
            int len = i - preSumMap.get(rem);
            maxLen = Math.max(maxLen, len);
        }
        // Store the first occurrence of the prefix sum
        if (!preSumMap.containsKey(sum)) {
            preSumMap.put(sum, i);
        }
    }
    return maxLen;
}
```

Complexity:

- **Time:** O(n)
- **Space:** O(n) — HashMap stores prefix sums

Takeaways:

- Prefix Sum + HashMap works for positive, negative, and zero values.
- Store the first occurrence of a prefix sum to obtain the longest possible subarray.

238. Product of Array Except Self

Prefix Sum | Array

Thought:

- Traverse from left to right maintainging prefixProd of nums in ans array
- Next traverse from right to left this time:: keep multiplying the suffixProd value of nums into the ans array
- Both prefixProd and suffixProd will be initialized with 1

Complexity:

- **Time:** O(n)
- **Space:** O(1) –prefix sums

Dry Run

```java
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] ans = new int[n];
        int prefixProd = 1;
        for(int i = 0; i < n; i++) {
            ans[i] = prefixProd;
            prefixProd *= nums[i];
        }
        int postfixProd = 1;
        for(int i = n - 1; i >= 0; i--) {
            ans[i] *= postfixProd;
            postfixProd *= nums[i];
        }
        return ans;
    }
}
```

Mark this:: first we have added the prefixProd and then multiplied the current value

Mark this:: And while coming from back we are multipliying postfixProd as well

#pattern

347. Top K Frequent Elements

Queue | Hash Map | Array

Approach: This problem has two part :

→ Maintain the frequency of each element [Easy]

→ Retriving the frequency for top k element [Hard]

Thought: Once you have the frequencies stored in your map, there are two primary ways to extract the top k elements:

1. *Min-Heap / PriorityQueue*

If you maintain a Min-Heap ordered by frequency and cap its size at k, the smallest frequency among the top elements will always be at the root.

How it works:

1. Create a `PriorityQueue<Integer>` configured to compare elements based on `numHs.get(a) - numHs.get(b)`.
2. Add each unique number from `numHs.keySet()`.
3. If the queue size exceeds k, remove the root (`poll()`).
4. At the end, the queue contains the k most frequent elements.

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> numHs = new HashMap<>();
        for (int num : nums) {
            numHs.merge(num, 1, Integer::sum);
        }
        // Min-heap ordered by frequency
        PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) ->
        numHs.get(a) - numHs.get(b));

        for (int num : numHs.keySet()) {
            heap.add(num);
            if (heap.size() > k) {
                heap.poll(); // remove least frequent element
            }
        }
        int[] result = new int[k];
        for (int i = 0; i < k; i++) {
            result[i] = heap.poll();
        }
        return result;
    }
}
```

Complexity:

- **Time:** O(n log k)
- **Space:** O(n + k)

#pattern

347. Top K Frequent Elements

Bucket Sort | Hash Map | Array

2. Bucket Sort

Since the maximum possible frequency of any element is n (the length of `nums`), you can create an array of lists (buckets) where the index represents the frequency, and each bucket stores the numbers that have that exact frequency.

How it works:

1. Create `List<Integer>[] buckets = new List[nums.length + 1]`.
2. Iterate through `numHs` entry set and place each element into `buckets[frequency]`.
3. Iterate from the highest bucket index down to 0, collecting elements until you have k elements.

Complexity:

- **Time:** O(n)
- **Space:** O(n)

Summary

- **Bucket Sort** is optimal here because frequencies are bounded by array length n, satisfying the follow-up requirement for linear time complexity.
- **Min-Heap** is intuitive and highly practical, especially for streaming data where the total length n isn't fixed in advance.

Dry Run

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> numsHs = new HashMap<>();
        for(int i = 0; i < nums.length; i++) {
            numsHs.merge(nums[i], 1, Integer::sum);
        }
        // Create buckets indexed by frequency
        List<Integer>[] bkt = new List[nums.length + 1];
        for(int key : numsHs.keySet()) {
            int freq = numsHs.get(key);
            if(bkt[freq] == null) {
                bkt[freq] = new ArrayList<>();
            }
            bkt[freq].add(key);
        }
        // Collect top k frequent elements starting from highest frequency bucket
        int[] res = new int [k];
        int ind = 0;
        for(int i = bkt.length - 1; i >= 0 && ind < k; i--) {
            if(bkt[i] != null) {
                for(int num : bkt[i]) {
                    res[ind] = num;
                    ind++;
                    if(ind == k) break;
                }
            }
        }
        return res;
    }
}
```

See how we can iterate hashMap w.r.t. key directly

#pattern

217. Contains Duplicate

Hash Map | Array

We have to check in the given array if any values occurs more than once.

Approach 1

→ Sort the array

→ Loop trough the array

→ Check if the just next element is same as last one, then return true

→ Return false in the end

```java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Arrays.sort(nums);
        for(int i = 0; i < nums.length - 1; i++) {
            if (nums[i] == nums[i + 1]) {
                return true;
            }
        }
        return false;
    }
}
```

Time Complexity → O(n log n)

Space Complexity → O(1)

Can be improved

Pro-Tip for Large Arrays

If you have a massive array and a multi-core processor, you should use `Arrays.parallelSort()` instead. It splits the array into sub-arrays, sorts them concurrently across available CPU threads, and merges them back together, drastically lowering the actual execution time. GitHub +3

Time Complexity Summary Table

Array Type	Algorithm Used	Best Case	Average Case	Worst Case
Primitives (`int[]`)	Dual-Pivot Quicksort	O(n)	$O(n \log n)$	$O(n^2)$ (or $O(n \log n)$ in modern fallback)
Objects (`Integer[]`)	Timsort	O(n)	$O(n \log n)$	$O(n \log n)$

#pattern

217. Contains Duplicate

Hash Map | Array

If we can use extra space then ::

Approach 2

→ Take a Set
→ Iterate in the array
  → Check If that value exist in the Set : If yes then return true
  → Else add that value into the Set
→ Come out of the for loop, and return false

```java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> numSet = new HashSet<>();
        for (int n : nums) {
            // add() returns false if the element already exists
            if (!numSet.add(n)) {
                return true;
            }
        }
        return false;
    }
}
```

```java
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> numSet = new HashSet<>();
        for(int n : nums) {
            if(numSet.contains(n)) return true;
            numSet.add(n);
        }
        return false;
    }
}
```

Your current code is logically correct with an optimal ***O(N)* time complexity**, but it can be made faster by removing redundant operations.

The Performance Issue

In your loop, you call `numSet.contains(n)` and then `numSet.add(n)`. This forces Java to calculate the hash code and search the internal buckets **twice** for every unique number.

The Optimization

In Java, the `Set.add()` method automatically checks for duplicates. It returns `false` if the element is already present in the set. You can combine both steps into a single line to cut the hashing operations in half.

Time Complexity → O(n)
Space Complexity → O(n)

#pattern

303. Range Sum Query - Immutable

Prefix Sum | Array

Leave this qns, let's focus on the pre-fix sum concept

Thought

Capturing a cumulative running total Sum for given array

Let's think index as days and the value as money spent on that particular day

0 1 2 3 4 5 ← *days*

e.g. spends {-2, 0, 3, -5, 2, -1}

Calculated prefixSum {-2, -2, 1, -4, -2, -3} ← *This will reflect the spend till date*

0 1 2 3 4 5 ← *days*

Now, since we have the prefixSum array, it's now easy to find the spends for any in between days

Suppose, someone asks: **"How much did you spend from Day 2 to Day 4?"**

Instead of summing days 2, 3, and 4 manually, we will take: Total up to Day 4 − Total up to Day 1

Because,

$$\underbrace{(\text{Day 0} + \text{Day 1} + \text{Day 2} + \text{Day 3} + \text{Day 4})}_{\text{Total up to Day 4}} - \underbrace{(\text{Day 0} + \text{Day 1})}_{\text{Total up to Day 1}} = \underbrace{\text{Day 2} + \text{Day 3} + \text{Day 4}}_{\text{Range Sum (2 to 4)}}$$

Time Complexity → O(n)

Space Complexity → O(1)

```java
class NumArray {
    int[] prefixSum;
    public NumArray(int[] nums) {
        prefixSum = new int[nums.length];
        prefixSum[0] = nums[0];
        for(int i = 1; i < nums.length; i++) {
            prefixSum[i] = prefixSum[i - 1] + nums[i];
        }
    }

    public int sumRange(int left, int right) {
        if(left == 0) return prefixSum[right];
        return prefixSum[right] - prefixSum[left - 1];
    }
}
```

#pattern

724. Find Pivot Index

Prefix Sum | Array

Thought

First calculate the sum of entire array and keep it in a variable
Also, keep a track of left side sum of array, initialize with 0
Then iterate in the loop again

- check if sum of left side == sum of right side => return index
- keep adding the current value in the variable which is there for saving left side of sum

```java
class Solution {
    public int pivotIndex(int[] nums) {
        int totalSum = 0;
        for(int n : nums) {
            totalSum += n;
        }
        int leftSum = 0;
        for(int i = 0; i < nums.length; i++) {
            if(leftSum == totalSum - leftSum - nums[i]) {
                return i;
            }
            leftSum += nums[i];
        }
        return -1;
    }
}
```

Take a Pen and do in paper, you will understand

Time Complexity → O(n)
Space Complexity → O(1)

#pattern

523. Continuous Subarray Sum (divisible by K)

Prefix Sum | Array

Thought

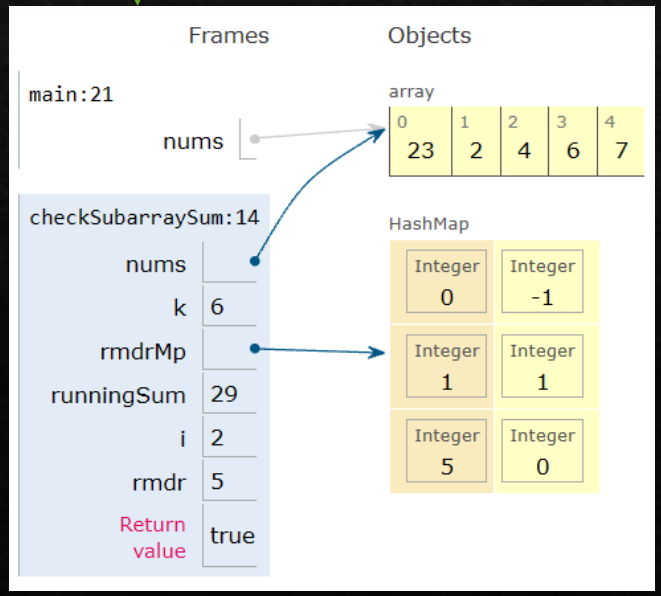
Simply, find all the subarrays and validate two checks as per given question.

But, in LC it will be givin TLE :p

To pass all test cases, use a **HashMap** to store the remainder of the prefix sum divided by k, mapped to its earliest index: Dry Run

```java
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        int n = nums.length;
        for(int i = 0; i < n; i++) {
            int sum = 0;
            for(int j = i; j < n; j++) {
                sum += nums[j];
                int len = j - i + 1;
                if(len >= 2 && sum % k == 0) return true;
            }
        }
        return false;
    }
}
```

```java
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        // Maps remainder -> earliest index it was seen
        Map<Integer, Integer> rmdrMp = new HashMap<>();
        // Base case: remainder 0 at index -1 to handle
        // subarrays starting from index 0 e.g. [2, 4]
        rmdrMp.put(0, -1);
        int runningSum = 0;
        for (int i = 0; i < nums.length; i++) {
            runningSum += nums[i];
            int rmdr = runningSum % k;
            if (rmdrMp.containsKey(rmdr)) {
                if (i - rmdrMp.get(rmdr) >= 2)
                    return true;
            } else rmdrMp.put(rmdr, i);
        }
        return false;
    }
}
```



Take a Pen and do in paper you will understand

Time Complexity → O(n)
Space Complexity → O(n)

#pattern

167. Two Sum II - Input Array Is Sorted

Binary Search | Two Pointers | Array

Thought

This problem can be solved with the same approach using HashMap how we did for TWO SUM, with a minimal change in the return statement because it's 1 based index. (Given in question)
But, since the given input is sorted, we implement binary search approach using two pointers, which will be the optimal solution for this problem with constant space.

Complexity

- **Time:** O(log n)
- **Space:** O(1)

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        int l = 0, h = nums.length - 1;
        while(l < h) {
            int sum = nums[l] + nums[h];
            if(sum == target) return new int[] {l + 1, h + 1};
            else if (sum > target) h--;
            else l++;
        }
        return new int[] {-1, -1};
    }
}
```

#pattern

11. Container With Most Water

Two Pointers | Array

Thought

Nice two pointer approach will easily solve this problem, just think of some real world scenario like:

1. Calculation the area i.e. height * width

Also, think properly on how to reduce the search space

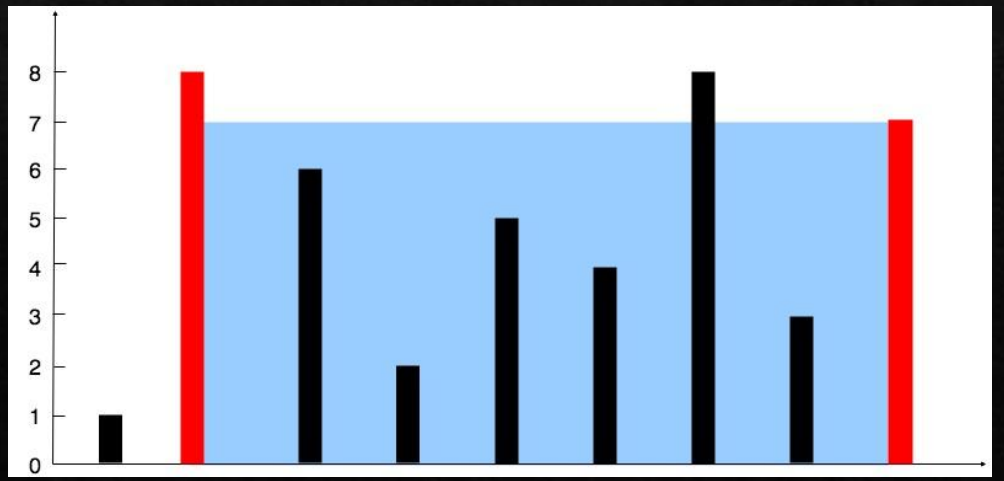
Complexity

- **Time:** O(n)
- **Space:** O(1)

Other than that it is really intresting how this solution is marked as medium, Also, very much curious on how to solve the GREEDY problems, like somehow it seem easy to me but yes, we need to think on how the serach space is reducing. Actually the image that you are looking in the right is from LC question only, It really gave me a good understanding of the problem and made it easier to think and come up with solution...

Don't worry bro, I am just practising touch typing, that is the reason I wrote this long passage. There is nothing so much WOW in this problem but definetly it's a good one!

```java
class Solution {
    public int maxArea(int[] height) {
        int l = 0;
        int r = height.length - 1;
        int ans = 0;
        while(l < r) {
            int lvl = Math.min(height[l], height[r]);
            ans = Math.max(ans, lvl * (r - l));
            if(height[l] < height[r]) l++;
            else r--;
        }
        return ans;
    }
}
```



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

#pattern

42. Trapping Rain Water

Two Pointers | Array

Thought

For each bar, the trapped water is calculated by the shorter of the highest bar to its left and the highest bar to its right, minus its own height:

$$\text{Water at } i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$$

Here, for determining the maxLeft / maxRight we are maintaining two arrays.

Complexity

- **Time:** O(3n)
- **Space:** O(2n)

There will be an optimal two pointers approach: DRY RUN HERE

For each step, move the pointer with the smaller bounding height inward, update its maximum height seen so far, and add the difference between that maximum and the current bar's height to the total water.

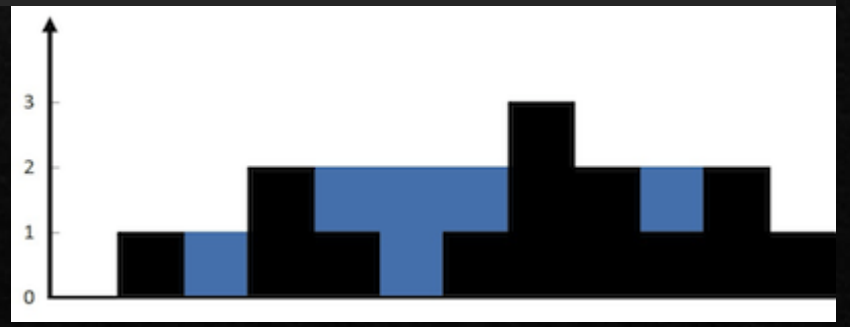
Complexity

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public int trap(int[] height) {
        int total = 0;
        int[] leftMax = new int[height.length];
        int[] rightMax = new int[height.length];
        leftMax[0] = height[0];
        for(int i = 1; i < height.length; i++) {
            leftMax[i] = Math.max(leftMax[i - 1], height[i]);
        }

        rightMax[height.length - 1] = height[height.length - 1];
        for(int i = height.length - 2; i >= 0; i--) {
            rightMax[i] = Math.max(rightMax[i + 1], height[i]);
        }

        for(int i = 0; i < height.length; i++) {
            if(height[i] < leftMax[i] && height[i] < rightMax[i]) {
                total += Math.min(leftMax[i], rightMax[i]) - height[i];
            }
        }
        return total;
    }
}
```



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

#pattern

2461. Maximum Sum of Distinct Subarrays With Length K

Sliding Window | Array

Thought

The main idea is to use a **sliding window**:

- Maintain a window that contains only distinct elements.
- Keep track of the current window sum.
- If a duplicate element appears, remove elements from the left until the window becomes valid again.
- Whenever the window size becomes exactly k, update the maximum sum.

Approach

Use a HashSet to store the elements currently present in the sliding window.

Maintain two pointers:

left → starting index of the window.

right → ending index of the window.

Traverse the array using the right pointer:

If nums[right] already exists in the set, remove elements from the left side until the duplicate is removed.

Add nums[right] to the set and add it to the current sum.

When the window size becomes k:

Compare the current sum with the maximum sum found so far.

Remove the leftmost element and move the window forward.

Return the maximum sum.

Dry Run

```java
class Solution {
    public long maximumSubarraySum(int[] nums, int k) {
        int n = nums.length;
        Set<Integer> set = new HashSet<>();
        long curSum = 0, maxSum = 0;
        int left = 0;
        for (int right = 0; right < n; right++) {
            while (set.contains(nums[right])) {
                set.remove(nums[left]);
                curSum -= nums[left];
                left++;
            }
            set.add(nums[right]);
            curSum += nums[right];
            if (right - left + 1 == k) {
                maxSum = Math.max(maxSum, curSum);
                set.remove(nums[left]);
                curSum -= nums[left];
                left++;
            }
        }
        return maxSum;
    }
}
```

Complexity

- **Time:** O(n)
- **Space:** O(k)

#pattern

704. Binary Search

Binary Search | Array

Problem:

- Given a **sorted array** nums.
- Find the index of target.
- Return -1 if target does not exist.
- Required time complexity: **O(log n)**.

Example:

Input: nums = [-1,0,3,5,9,12], target = 9, Output: 4
Input: nums = [-1,0,3,5,9,12], target = 2, Output: -1

Approach:

- Use **two pointers**: low and high.
- Find middle element.
- If middle < target → search right half.
- If middle > target → search left half.
- Repeat until found or range is invalid.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[mid] < target) low = mid + 1;
            else if(nums[mid] > target) high = mid - 1;
            else return mid;
        }
        return -1;
    }
}
```

Takeaways:

- Works only on **sorted arrays**
- Divides search space by half each step
- Core pattern for **logarithmic search problems**
- Foundation for many advanced variations

#pattern

35. Search Insert Position / Implement Lower Bound → *smallest index such that* arr[ind] >= target

Binary Search | Array

Problem_V0:

- Given a **sorted array of distinct integers**.
- Return index of target if found.
- Otherwise, return the **position where it should be inserted**.

Example_V0:

Input: [1,3,5,6], target = 5, Output: 2
Input: [1,3,5,6], target = 2, Output: 1
Input: [1,3,5,6], target = 7, Output: 4

Problem_V1:

- Given a **sorted array** arr.
- Find the **smallest index** where arr[i] ≥ target.
- If no such element exists, return **array length**.

Example_V1:

Input: [2,3,7,10,11,11,25], target = 9, Output: 3
Input: [2,3,7,10,11,11,25], target = 11, Output: 4
Input: [2,3,7,10,11,11,25], target = 100, Output: 7

```java
class Solution {
    int lowerBound(int[] arr, int target) {
        for(int i = 0; i < arr.length; i++) {
            if(arr[i] >= target) return i;
        }
        return arr.length;
    }
}
```

Thought:

- We have to find the index where target value should be inserted
- Since, the given array is sorted we can traverse the entire array and check if the current value is greater than target, then it's the index.
- If we never find any such value, it means the target is bigger than max value, so add in the end

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Can be improved

#pattern

35. Search Insert Position / Implement Lower Bound

Binary Search | Array

This is to make sure, if no value found in the array which is lesser or equal then we need to add into the last index

Thought:

- Initialize a variable ans with value as length of the array
- Start the basic BS templet, Here if we find the value more or equal then target, then simply update the ans with mid, and move the search space to left else move it to right.

Approach:

- Apply binary search on sorted array.
- Track possible answer when condition is met.
- Move left to find **smallest valid index**.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Takeaways:

- Lower bound finds **first valid position**
- Binary search works beyond exact matches
- Useful for **range queries & insertion points**
- Foundation for **upper bound problems**

Dry Run

```java
class Solution {
    int lowerBound(int[] arr, int target) {
        int ans = arr.length;
        int low = 0, high = arr.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(arr[mid] >= target) {
                ans = mid;
                high = mid - 1;
            } else low = mid + 1;
        }
        return ans;
    }
}
```

#pattern

Implement Upper Bound → *smallest index such that* arr[ind] > target

Binary Search | Array

Problem:

- Given a **sorted array** arr.
- Find the **smallest index** where arr[i] > target.
- If no such element exists, return **array length**.

Example:

Input: [2,3,7,10,11,11,25], target = 9, Output: 3
Input: [2,3,7,10,11,11,25], target = 11, Output: 6
Input: [2,3,7,10,11,11,25], target = 100, Output: 7

Approach:

- Use **binary search** on the sorted array.
- When arr[mid] > target, store index as answer.
- Move left to find the **smallest valid index**.
- If not found, return array length.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

```java
class Solution {
    int upperBound(int[] arr, int target) {
        int ans = arr.length;
        int low = 0, high = arr.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(arr[mid] > target) {
                ans = mid;
                high = mid - 1;
            } else low = mid + 1;
        }
        return ans;
    }
}
```

Takeaways:

- Upper bound finds **first strictly greater element**
- Differs from lower bound by > vs ≥
- Useful for **frequency counting & ranges**
- Core **binary search variant**

34. Find First and Last Position of Element

Binary Search | Array

Problem:

- Given a **sorted array** nums.
- Find the **starting and ending index** of target.
- If target doesn't exist, return [-1, -1].

Example:

Input: [5,7,7,8,8,10], target = 8, Output: [3,4]

Input: [], target = 0, Output: [-1,-1]

Input: [5,7,7,8,8,10], target = 6, Output: [-1,-1]

Approach:

- Traverse the array.
- Record first and last index where target appears.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

Can be improved

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int ans[] = {-1, -1};
        for(int i = 0; i < nums.length; i++) {
            if(nums[i] == target) {
                if(ans[0] == -1) {
                    ans[0] = i;
                    ans[1] = i;
                } else {
                    ans[1] = i;
                }
            }
        }
        return ans;
    }
}
```

#pattern

34. Find First and Last Position of Element

Binary Search | Array

Approach:

- Find **lower bound** of target → first occurrence.
- Find **upper bound** of target → index after last occurrence.
- Final range = [lowerBound, upperBound - 1].

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Takeaways:

- Combines **lower bound + upper bound**
- Binary search avoids linear scanning
- Important pattern for **range queries**
- Very common **interview question**

These checks are very important here, it is the key areas to handle edge case

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int lb = lowerBound(nums, target);
        if(lb == nums.length || nums[lb] != target)
            return  new int[]{-1, -1};
        return new int[]{lb, upperBound(nums, target) - 1};
    }

    private int lowerBound(int[] arr, int target) {
    ...
    }

    private int upperBound(int[] arr, int target) {
    ...
    }
}
```

#pattern

33. Search in Rotated Sorted Array

Binary Search | Array

Problem:

- Given a **sorted array with distinct elements**, rotated at an unknown index.
- Search for target and return its index.
- Return -1 if target does not exist.

Example:

Input: [4,5,6,7,0,1,2], target = 0, Output: 4
Input: [4,5,6,7,0,1,2], target = 3, Output: -1
Input: [1], target = 0, Output: -1

Approach:

- Use binary search with rotation awareness.
- Identify which half is **sorted** at each step.
- Check if target lies in the sorted half.
- Narrow search space accordingly.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int search(int[] nums, int target) {
        int low = 0, high = nums.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[mid] == target) return mid;
            if(nums[low] <= nums[mid]){
                if(nums[low]<= target && target <= nums[mid]){
                    high = mid - 1;
                } else low = mid + 1;
            } else {
                if(nums[mid] <= target && target <= nums[high]){
                    low = mid + 1;
                } else high = mid - 1;
            }
        }
        return -1;
    }
}
```

Takeaways:

- One half is **always sorted** in rotated arrays
- Identify the sorted half then decide direction using sorted half
- Classic **binary search variation**
- Key pattern for rotated-array problems

#pattern

81. Search in Rotated Sorted Array II

Binary Search | Array

Problem:

- Given a **sorted array (may contain duplicates)** rotated at an unknown index.
- Determine if target exists in the array.
- Return true if found, else false.
- Minimize number of operations.

Example:

Input: [2,5,6,0,0,1,2], target = 0, Output: true
Input: [2,5,6,0,0,1,2], target = 3, Output: false
Input: [1], target = 0, Output: -1

Approach:

- Use binary search with rotation handling.
- If nums[low] == nums[mid] == nums[high]:
  - Shrink search space (low++, high--)
- Otherwise, identify the **sorted half**.
- Check if target lies in sorted half and adjust bounds.

Complexity:

- **Time:** O(log n) (worst case O(n) due to duplicates)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public boolean search(int[] nums, int target) {
    int low = 0, high = nums.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[mid] == target) return true;
            if(nums[mid] == nums[low] && nums[mid] == nums[high]){
                low = low + 1;
                high = high - 1;
                continue;
            }
            if(nums[low] <= nums[mid]){
                if(nums[low]<= target && target <= nums[mid]){
                    high = mid - 1;
                } else low = mid + 1;
            } else {
                if(nums[mid] <= target && target <= nums[high]){
                    low = mid + 1;
                } else high = mid - 1;
            }
        }
        return false;
    }
}
```

Takeaways:

- Duplicates break strict binary search guarantees
- Shrinking bounds helps resolve ambiguity
- Extension of **rotated array search**
- Important edge-case handling problem

153. Find Minimum in Rotated Sorted Array

Binary Search | Array

Problem:

- Given a **sorted array of unique elements**, rotated multiple times.
- Find the **minimum element**.

Example:

Input: [3,4,5,1,2], Output: 1
Input: [4,5,6,7,0,1,2], Output: 0
Input: [11,13,15,17], Output: 11

Approach:

- Follow same BS template, but with some mindful tweak
- Check for the min value, then move to the opposite side

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int findMin(int[] nums) {
        int low = 0, high = nums.length-1, min = Integer.MAX_VALUE;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[low] <= nums[mid]){
                min = Math.min(min, nums[low]);
                low = mid + 1;
            } else {
                min = Math.min(min, nums[mid]);
                high = mid - 1;
            }
        }
    return min;
    }
}
```

Takeaways:

- Rotated array always has **one unsorted half**
- Minimum lies at the **rotation point**
- Binary search adapts to rotation
- Core problem for **rotated-array patterns**

#pattern

Find Kth Rotation

Binary Search | Array

Problem:

- Given a **sorted array of distinct integers**, right-rotated k times.
- Find the value of k (number of rotations).
- If array is not rotated, return 0.

Example:

Input: [5,1,2,3,4], Output: 1
Input: [1,2,3,4,5], Output: 0

Approach:

- Traverse array.
- Find first index where arr[i] > arr[i+1].
- Rotation count = i + 1.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public int findKRotation(int arr[]) {
        int n = arr.length;
        for(int i = 0; i < n - 1; i++) {
            if(arr[i] > arr[i + 1]) {
                return i + 1;
            }
        }
        return 0;
    }
}
```

Find Kth Rotation

Binary Search | Array

Approach:

- Rotation count equals index of **minimum element**.
- Use binary search to locate minimum.
- Compare mid with high to decide search space.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Takeaways:

- Rotation count = **index of minimum element**
- Binary search works due to **sorted halves**
- Extension of **Find Min in Rotated Array**
- Key rotation-related pattern

Dry Run

```java
class Solution {
    public int findKRotation(int arr[]) {
        int low = 0;
        int high = arr.length - 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] > arr[high]) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }
}
```

540. Single Element in a Sorted Array

Binary Search | Array

Problem:

- Given a **sorted array.**
- Every element appears **twice**, except **one element**.
- Find the element that appears **only once**.

Example:

Input: [1,1,2,3,3,4,4,8,8], Output: 2
Input: [3,3,7,7,10,11,11], Output: 10

Approach:

- In a balanced array:
  - Pairs start at **even index** and end at **odd index**.
- The unique element **breaks this pattern**.
- Use binary search:
  - Check index parity and neighboring values.
  - Decide which half still follows the pairing pattern.
- Shrink search space until the single element is found.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int singleNonDuplicate(int[] nums) {
        int len = nums.length;
        if(len == 1) return nums[0];
        if(nums[0] != nums[1]) return nums[0];
        if(nums[len - 1] != nums[len - 2]) return nums[len - 1];
        int low = 1, high = len - 2;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[mid] != nums[mid - 1] && nums[mid] != nums[mid + 1])
            return nums[mid];
            if((mid % 2 == 1 && nums[mid - 1] == nums[mid]) ||
               (mid % 2 == 0 && nums[mid + 1] == nums[mid])) {
                low = mid + 1;
               } else high = mid - 1;
        }
        return -1;
    }
}
```

Takeaways:

- Index parity is the key observation
- Sorted structure enables binary search
- Pattern breaks reveal the answer
- Classic **binary search on indices**

162. Find Peak Element

Binary Search | Array

Problem:

- A **peak element** is greater than its neighbors.
- Given an array nums, find **any peak index**.
- Assume nums[-1] = nums[n] = -∞.
- Multiple peaks may exist.

Example:

Input: [1,2,3,1], Output: 2

Input: [1,2,1,3,5,6,4], Output: 1 or 5

Approach:

- Check each element.
- If greater than both neighbors → peak found.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public int findPeakElement(int[] nums) {
        for(int i = 0; i < nums.length; i++) {
            if((i == 0 || nums[i - 1] < nums[i]) &&
              ((i == nums.length - 1) || nums[i] > nums[i + 1])
            ) return i;
        }
        return 0;
    }
}
```

162. Find Peak Element

Binary Search | Array

Approach:

- Compare middle element with neighbors.
- If increasing → peak lies on right side.
- If decreasing → peak lies on left side.
- Continue until a peak is found.

Complexity:

- **Time:** O(log n)
- **Space:** O(1)

Takeaways:

- Peak always exists due to -∞ boundaries
- No need to find global maximum
- Binary search works on **trend direction**
- Classic **divide & conquer** problem

Dry Run

```java
class Solution {
    public int findPeakElement(int[] nums) {
        int n = nums.length;
        if(n == 1) return 0;
        if(nums[0] > nums[1]) return 0;
        if(nums[n - 1] > nums[n - 2]) return n - 1;
        int low = 1, high = n - 2;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(nums[mid] > nums[mid - 1] && nums[mid] > nums[mid + 1])
            return mid;
            else if(nums[mid] > nums[mid - 1])
            low = mid + 1;
            else high = mid - 1;
        }
        return 0;
    }
}
```

69. Sqrt(x)

Binary Search | Array

Problem:

- Given a **non-negative integer x**.
- Return **⌊√x⌋** (square root rounded down).
- Do **not** use built-in power functions.

Example:

Input: x = 4, Output: 2
Input: x = 8, Output: 2

Approach:

- Try all integers from 1 to x.
- Track largest i such that i * i ≤ x.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public int mySqrt(int x) {
        int ans = 0;
        for(long i = 1; i <= x; i++) {
            if(i * i <= x) ans = (int)i;
            else break;
        }
        return ans;
    }
}
```

69. Sqrt(x)

Binary Search | Array

Approach:

- Search range: 1 to x.
- For each mid:
  - If mid * mid ≤ x, store answer and move right.
  - Else, move left.
- Final stored value is the result.

Complexity:

- **Time:** O(x)
- **Space:** O(1)

Takeaways:

- Square root problem = **monotonic function**
- Binary search avoids overflow & inefficiency
- Use long for mid * mid
- Classic **binary search on answer**

Dry Run

```java
class Solution {
    public int mySqrt(int x) {
        int ans = 0;
        long low = 1, high = x;
        while(low <= high) {
            long mid = (low + high) / 2;
            if(mid * mid <= x) {
                ans = (int)mid;
                low = mid + 1;
            } else high = mid - 1;
        }
        return ans;
    }
}
```

Find nth root of m

Binary Search | Array

Problem:

- Given two integers n and m.
- Find the **integer nth root** of m.
- If no integer root exists, return -1.

Example:

Input: n = 3, m = 8, Output: 2
Input: n = 3, m = 9, Output: -1
Input: n = 4, m = 16, Output 2

Approach:

- Search in range 1 to m.
- For each mid:
  - Compute mid^n.
  - If equal to m → answer found.
  - If greater → search left.
  - If smaller → search right.
- If no exact match, return -1.

Complexity:

- **Time:** O(log m)
- **Space:** O(1)

```java
class Solution {
    public int nthRoot(int n, int m) {
        if(m == 0) return 0;
        int low = 1, high = m;
        while(low <= high) {
            int mid = (high + low) / 2;
            if(Math.pow(mid, n) == m) return mid;
            else if(Math.pow(mid, n) > m) high = mid - 1;
            else low = mid + 1;
        }
        return - 1;
    }
}
```

Takeaways:

- Root problems are **monotonic** → binary search applies
- Avoid brute force for large m
- Careful with **overflow** while computing powers
- Classic **binary search on answer** pattern

875. Koko Eating Bananas

Binary Search | Array

Problem:

- Given banana piles piles[] and h hours.
- Koko eats k bananas per hour from one pile.
- Find **minimum integer k** to finish all bananas within h hours.

Example:

Input: [3,6,7,11], h = 8, Output: 4
Input: [30,11,23,4,20], h = 5, Output: 30
Input: [30,11,23,4,20], h = 6, Output 23

Approach:

- Eating speed k lies between 1 and max(piles).
- For a given k, compute total hours needed.
- If hours > h → speed too slow → increase k.
- Else → try smaller k.
- Final k is the minimum valid speed.

Takeaways:

- Classic **binary search on answer**
- Problem has **monotonic behaviour**
- Ceiling division is crucial
- Common scheduling / optimization pattern

You know why returned *low*
While implementing BS on answers, the pivot point lies into the opposite Polarity. Suppose, possible answers started from low side, then the answer will always be on the high at the end of BS iteration and vice versa...

Dry Run

```java
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int low = 1;
        int high = maxi(piles);
        while(low <= high) {
            int mid = (low + high) / 2;

            if(finishTime(piles, mid) > h) {
                low = mid + 1;
            } else high = mid - 1;
        }
        return low;
    }
    public int maxi(int[] arr) {
        int max = Integer.MIN_VALUE;
        for(int ele : arr) {
            if(ele > max) {
                max = ele;
            }
        }
        return max;
    }
    public int finishTime(int[] arr, int hrs) {
        int totalhrs = 0;
        for (int j : arr) {
            totalhrs += Math.ceil((double)j / (double)(hrs));
        }
        return totalhrs;
    }
}
```

Complexity:

- **Time:** O(n log max(piles))
- **Space:** O(1)

#pattern

1482. Minimum days to make M bouquets

Binary Search | Array

Problem:

- Given bloomDay[], m bouquets, and k adjacent flowers per bouquet.
- Each flower can be used **once** and only after it blooms.
- Find the **minimum days** to make m bouquets.
- Return -1 if impossible.

Example:

Input: [1,10,3,10,2], m = 3, k = 1, Output: 3
Input: [1,10,3,10,2], m = 3, k = 2, Output: -1
Input: [7,7,7,7,12,7,7], m = 2, k = 3, Output: 12

Approach:

- Try each day from 0 to max(bloomDay).
- Check if m bouquets can be formed using bloomed flowers.
- Stop at first valid day.

Complexity:

- **Time:** O(n * maxDay)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        for(int i = 0; i <= getMaxi(bloomDay); i++) {
            if(isPossible(bloomDay, i, m, k) == true)
            return i;
        }
        return -1;
    }
    public boolean isPossible(int[] arr, int day, int m, int k) {
        int cnt = 0, noOfB = 0;
        for(int i = 0; i < arr.length; i++) {
            if(arr[i] <= day) {
                cnt++;
            } else{
                noOfB += cnt/k;
                cnt = 0;
            }
        }
        noOfB += cnt/k;
        if(noOfB >= m) return true;
        else return false;
    }
    public int getMaxi(int[] arr) {
        int max = arr[0];
        for(int i = 1; i < arr.length; i++) {
            if(arr[i] > max) max = arr[i];
        }
        return max;
    }
}
```

1482. Minimum days to make M bouquets

Binary Search | Array

Approach:

- Search days between min(bloomDay) and max(bloomDay).
- For a given day:
  - Count consecutive bloomed flowers.
  - Convert them into bouquets (count / k).
- If possible → try smaller days.
- Else → increase days.

Complexity:

- **Time:** O(n log maxDay)
- **Space:** O(1)

Takeaways:

- Classic **binary search on answer**
- Feasibility check drives optimization
- Adjacent grouping is crucial
- Common in **allocation & scheduling** problems

```java
class Solution {
    public int minDays(int[] bloomDay, int m, int k) {
        int low = getMini(bloomDay);
        int high = getMaxi(bloomDay);
        int ans = -1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(isPossible(bloomDay, mid, m, k) == true){
                ans = mid;
                high = mid - 1;
            }
            else low = mid + 1;
        }
        return ans;
    }
    public boolean isPossible(int[] arr, int day, int m, int k) {
        int cnt = 0, noOfB = 0;
        for(int i = 0; i < arr.length; i++) {
            if(arr[i] <= day) {
                cnt++;
            } else{
                noOfB += cnt/k;
                cnt = 0;
            }
        }
        noOfB += cnt/k;
        if(noOfB >= m) return true;
        else return false;
    }
}
```

Find the Smallest Divisor Given a Threshold

Binary Search | Array

Problem:

- Given an array nums and an integer threshold.
- Choose a **positive divisor d**.
- Compute ceil(nums[i] / d) for each element and sum them.
- Find the **smallest d** such that the sum ≤ threshold.

Example:

Input: [1,2,5,9], threshold = 6, Output: 5
Input: [44,22,33,11,1], threshold = 5, Output: 44

Approach:

- Try every divisor from 1 to max(nums).
- For each divisor, compute the ceiling-sum.
- Return first divisor satisfying the condition.

Complexity:

- **Time:** O(n * max(nums))
- **Space:** O(1)

```java
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int max = Integer.MIN_VALUE;
        for(int i = 0; i < nums.length; i++) {
           max = Math.max(max, nums[i]);
        }
            for(int div = 1; div <= max; div++) {
                int sum = 0;
            for(int i = 0; i < nums.length; i++) {
                sum += Math.ceil((double)(nums[i]) / (double)(div));
            }
            if(sum <= threshold) return div;
            }
            return max;
        }
    }
```

Find the Smallest Divisor Given a Threshold

Binary Search | Array

Approach:

- Search divisor range 1 to max(nums).
- For a mid divisor:
  - Compute ceiling-sum.
  - If sum ≤ threshold → divisor is **possible**, try smaller.
  - Else → divisor too small, increase it.
- Final answer is the smallest valid divisor (low).

Complexity:

- **Time:** O(n log max(nums))
- **Space:** O(1)

Takeaways:

- Another classic **binary search on answer**
- Larger divisor → **smaller sum**
- Ceil division trick: **(a + d - 1) / d**
- Returning low works due to **polarity flip** in binary search

*#Pro Tip: Initially find the ceiling value using Math.ceil and then use division trick, saying It's ssignificantly **faster** as it avoids expensive floating-point conversions and method calls.*

Dry Run

```java
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int low = 1, high = maxi(nums);
        while(low <= high) {
            int mid = (low + high) / 2;
            if(sumOfDiv(nums, mid) <= threshold) high = mid - 1;
            else low = mid + 1;
        }
        return low;
    }
    public int maxi(int[] arr) {
        int max = arr[0];
        for(int n : arr) {
            if(n > max) max = n;
        }
        return max;
    }
    public int sumOfDiv(int[] nums, int d) {
        int sum = 0;
        for(int n : nums) {
            sum += ceil(n, d);
        }
        return sum;
    }
    public int ceil(int a, int d) {
        return (a + d - 1) / d;
    }
}
```

Capacity To Ship Packages Within D Days

Binary Search | Array

Problem:

- Given package weights on a conveyor belt.
- Packages must be shipped **in order** within days.
- Ship has a **fixed capacity per day**.
- Find the **minimum ship capacity** to meet the deadline.

Example:

Input: [1,2,3,4,5,6,7,8,9,10], days = 5, Output: 15
Input: [3,2,2,4,1,4], days = 3, Output: 6
Input: [1,2,3,1,1], days = 4, Output: 3

Approach:

- Capacity range:
  - Minimum = max(weights)
  - Maximum = sum(weights)
- Try every capacity in range.
- Compute required days for each capacity.
- Return first valid capacity.

Complexity:

- **Time:** O(n * (sum - max))
- **Space:** O(1)

```java
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int[] range = getRange(weights);
        for(int i = range[0]; i <= range[1]; i++) {
            int daysNeeded = daysReq(weights, i);
            if(daysNeeded <= days) return i;
        }
        return range[1];
    }
    public int[] getRange(int[] wt){
        int max = Integer.MIN_VALUE;
        int sum = 0;
        for(int w : wt){
            if(w > max) max = w;
            sum += w;
        }
        return new int[] {max, sum};
    }
    public int daysReq(int[] wt, int cap){
        int days = 1, load = 0;
        for(int w : wt) {
            if(load + w > cap) {
                days++;
                load = w;
            } else load += w;
        }
        return days;
    }
}
```

Capacity To Ship Packages Within D Days

Binary Search | Array

Approach:

- Search capacity between max(weights) and sum(weights).
- For a given capacity:
  - Simulate loading packages day-wise.
- If required days ≤ given days → try smaller capacity.
- Else → increase capacity.

Complexity:

- **Time:** O(n log(sum))
- **Space:** O(1)

Takeaways:

- Another **binary search on answer** problem
- Capacity vs days shows **monotonic behavior**
- Must preserve **package order**
- Common logistics & scheduling pattern

#Pro Tip: Binary search works on monotonic polarity. After the polarity flips, low always points to the first valid answer.

Dry Run

```java
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int[] range = getRange(weights);
        int low = range[0], high = range[1];
        while(low <= high) {
            int mid = (low + high) / 2;
            if(daysReq(weights, mid) <= days){
                high = mid - 1;
            } else low = mid + 1;
        }
        return low;
    }
    public int[] getRange(int[] wt){
        int max = Integer.MIN_VALUE;
        int sum = 0;
        for(int w : wt){
            if(w > max) max = w;
            sum += w;
        }
        return new int[] {max, sum};
    }
    public int daysReq(int[] wt, int cap){
        int days = 1, load = 0;
        for(int w : wt) {
            if(load + w > cap) {
                days++;
                load = w;
            } else load += w;
        }
        return days;
    }
}
```

1539. Kth Missing Positive Number

Binary Search | Array

Problem:

- Given a **strictly increasing sorted array** of positive integers.
- Find the **k-th missing positive number**.
- Missing numbers are counted from 1 onwards.

Example:

Input: [2,3,4,7,11], k = 5, Output: 9
Input: [1,2,3,4], k = 2, Output: 6
Input: [3,4,5], k = 2, Output: 2

Approach:

- Iterate through array.
- If current element ≤ k, increment k.
- Final k gives the answer.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public int findKthPositive(int[] arr, int k) {
        for (int num : arr) {
            if (num <= k) k++;
            else break;
        }
        return k;
    }
}
```

1539. Kth Missing Positive Number

Binary Search | Array

Approach:

- Missing count at index i:
  - missing = arr[i] - (i + 1)
- Binary search to find first index where missing ≥ k.
- Answer = low + k.

Complexity:

- **Time:** O(n log(sum))
- **Space:** O(1)

Takeaways:

- Formula arr[i] - (i+1) is key insight
- Binary search on **missing count**
- Elegant math + search combo
- Common interview favourite

#Pro Tip: These kind of intuition will never come in the Interview all of sudden, so practice and revision more & more.

Dry Run

```java
class Solution {
    public int findKthPositive(int[] arr, int k) {
        int low = 0, high = arr.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            int missing = arr[mid] - (mid + 1);
            if(missing < k) low = mid + 1;
            else high = mid - 1;
        }
        return low + k;
    }
}
```

Aggressive Cows

Binary Search | Array

Problem:

- Given stall positions stalls[] (unique).
- Place k cows in stalls such that the **minimum distance between any two cows is maximized**.
- Return that **maximum possible minimum distance**.

Example:

Input: [1,2,4,8,9], k = 3, Output: 3
Input: [10,1,2,7,5], k = 3, Output: 4
Input: [2,12,11,3,26,7], k = 5, Output: 1

Approach:

- Sort stall positions.
- Try all possible distances from 1 to maxDist.
- For each distance, greedily place cows.
- Track the largest valid distance.

Complexity:

- **Time:** O(n * maxDist)
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int aggressiveCows(int[] stalls, int k) {
        int ans = 0;
        Arrays.sort(stalls);
        int maxDist = stalls[stalls.length - 1] - stalls[0];
        for(int d = 1; d <= maxDist; d++) {
            if(canPlaced(stalls, k, d)) {
                ans = d;
            }
        }
        return ans;
    }
    public boolean canPlaced(int[] stall, int cows, int dist) {
        int curCnt = 1, last = stall[0];
        for(int i = 1; i < stall.length; i++) {
            if(stall[i] - last >= dist) {
                curCnt++;
                last = stall[i];
            }
            if(curCnt >= cows) return true;
        }
        return false;
    }
}
```

Aggressive Cows

Binary Search | Array

Approach:

- Sort stalls.
- Search distance between 1 and max(stalls) - min(stalls).
- For a candidate distance:
  - Greedily place cows with at least that gap.
- If placement is possible → try larger distance.
- Else → reduce distance.

Complexity:

- **Time:** O(n log maxDist)
- **Space:** O(1)

Takeaways:

- Classic **binary search on answer**
- Greedy check validates feasibility
- Sorting is mandatory
- Foundational problem for **maximize minimum** patterns

#Pro Tip: It's crucial to get the brute force solution, once that is done, optimizing to Binary search is easy.

```java
class Solution {
    public int aggressiveCows(int[] stalls, int k) {
        Arrays.sort(stalls);
        int low = 1;
        int high = stalls[stalls.length - 1] - stalls[0];
        while(low <= high) {
            int mid = (low + high) / 2;
            if(canPlaced(stalls, k, mid)){
                low = mid + 1;
            } else high = mid - 1;
        }
        return high;
    }
    public boolean canPlaced(int[] stall, int cows, int dist) {
        int curCnt = 1, last = stall[0];
        for(int i = 1; i < stall.length; i++) {
            if(stall[i] - last >= dist) {
                curCnt++;
                last = stall[i];
            }
            if(curCnt >= cows) return true;
        }
        return false;
    }
}
```

Allocate Min Pages / Split Array Largest Sum*

Binary Search | Array

Problem:

- Given an array arr[] where arr[i] = pages in the *i-th* book.
- Allocate books to k students such that:
  - Each student gets **at least one book**.
  - Books are assigned **contiguously**.
- Minimize the **maximum pages** assigned to any student.
- Return -1 if allocation is impossible.

Example:

Input: [12,34,67,90], k = 2, Output: 113
Input: [15,17,20], k = 5, Output: -1

Approach:

- Minimum possible pages = max(arr)
- Maximum possible pages = sum(arr)
- Try every value in this range.
- For each value, check how many students are required.
- First valid value is the answer.

Complexity:

- **Time:** O(n * (sum - max))
- **Space:** O(1)

Dry Run

```java
class Solution {
    public int findPages(int[] arr, int k) {
        if (arr.length < k) return -1;
        int low = 0, high = 0;
        for (int pages : arr) {
            low = Math.max(low, pages);
            high += pages;
        }
        for (int pages = low; pages <= high; pages++) {
            if (cntStd(arr, pages) <= k) {
                return pages;
            }
        }
        return -1;
    }
    public int cntStd(int[] arr, int limit) {
        int students = 1, currentSum = 0;
        for (int pages : arr) {
            if (currentSum + pages <= limit) {
                currentSum += pages;
            } else {
                students++;
                currentSum = pages;
            }
        }
        return students;
    }
}
```

Allocate Min Pages / Split Array Largest Sum*

Binary Search | Array

Approach:

- Search space: max(arr) → sum(arr).
- For a given page limit:
  - Count number of students needed.
- If students ≤ k → try smaller limit.
- Else → increase limit.

Complexity:

- **Time:** O(n log(sum))
- **Space:** O(1)

Takeaways:

- Classic **binary search on answer**
- Greedy validation using contiguous grouping
- Impossible when students > books
- Foundation problem for **allocation optimization**

#Pro Tip: In Interview, try to build the intuition. There might be some variation in the question. You have to observe the pattern wisely.

This problem also has the exactly same thought process: **410. Split Array Largest Sum*

Dry Run

```java
class Solution {
    public int findPages(int[] arr, int k) {
        if (arr.length < k) return -1;
        int low = 0, high = 0;
        for (int pages : arr) {
            low = Math.max(low, pages);
            high += pages;
        }
        int result = -1;
        while (low <= high) {
            int mid = (high + low) / 2;
            if (cntStd(arr, mid) <= k) {
                result = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return result;
    }
    public int cntStd(int[] arr, int pages) {
        int std = 1, currentSum = 0;
        for (int bookPages : arr) {
            if (currentSum + bookPages <= pages) {
                currentSum += bookPages;
            } else {
                std++;
                currentSum = bookPages;
            }
        }
        return std;
    }
}
```

4. Median of Two Sorted Arrays

Binary Search | Array

Problem:

- Given two sorted arrays nums1 and nums2.
- Return the **median** of the combined sorted array.
- Required time complexity: **O(log (m+n))**.

Example:

Input: [1,3], [2], Output: 2.0
Input: [1,2], [3,4], Output: 2.5

Approach:

- Merge both arrays into a new sorted list.
- Compute median from merged array.

Complexity:

- **Time:** O(m+n)
- **Space:** O(m+n)

Dry Run

```java
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        List<Integer> merged = new ArrayList<>();
        int i = 0, j = 0;
        while(i < nums1.length && j < nums2.length) {
            if(nums1[i] < nums2[j]){
                merged.add(nums1[i++]);
            } else merged.add(nums2[j++]);
        }
        while(i < nums1.length) {
            merged.add(nums1[i++]);
        }
        while(j < nums2.length) {
            merged.add(nums2[j++]);
        }
        int n = merged.size();
        if (n % 2 == 1)
            return merged.get(n / 2);
        return (merged.get(n / 2 - 1) + merged.get(n / 2)) / 2.0;
    }
}
```

#pattern

4. Median of Two Sorted Arrays

Binary Search | Array

Approach:

- Traverse both arrays like merge step.
- Stop when median positions are reached.
- Avoid storing full merged array.

Complexity:

- **Time:** O(m+n)
- **Space:** O(1)

#Pro Tip: In Interview, try to build the intuition. There might be some variation in the question. You have to observe the pattern wisely.

Dry Run

```java
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        int n1 = nums1.length, n2 = nums2.length;
        int n = n1 + n2;
        int ind2 = n / 2;
        int ind1 = ind2 - 1;
        int cnt = 0, i = 0, j = 0;
        int ind1el = -1, ind2el = -1;
        while(i < n1 && j < n2) {
            if(nums1[i] < nums2[j]){
                if(cnt == ind1) ind1el = nums1[i];
                if(cnt == ind2) ind2el = nums1[i];
                cnt++;
                i++;
            } else {
                if(cnt == ind1) ind1el = nums2[j];
                if(cnt == ind2) ind2el = nums2[j];
                cnt++;
                j++;
            }
        }
        while(i < n1) {
            if(cnt == ind1) ind1el = nums1[i];
                if(cnt == ind2) ind2el = nums1[i];
                cnt++;
                i++;
        }
        while(j < n2) {
            if(cnt == ind1) ind1el = nums2[j];
                if(cnt == ind2) ind2el = nums2[j];
                cnt++;
                j++;
        }
        if (n % 2 == 1) return (double) ind2el;
        return (ind1el + ind2el) / 2.0;
    }
}
```

4. Median of Two Sorted Arrays

Binary Search | Array

Approach:

- Always binary search on **smaller array**.
- Partition both arrays so:
  - Left half size = Right half size (or +1)
  - All left elements ≤ all right elements.
- Median derived from partition boundaries.

Complexity:

- **Time:** O(log(min(m,n)))
- **Space:** O(1)

Takeaways:

- Hard problem with elegant partition logic
- Key idea: **balanced partition**
- Always binary search on smaller array
- One of the most famous **binary search applications**

Dry Run

```java
class Solution {
public double findMedianSortedArrays(int[] a, int[] b) {
    int n1 = a.length;
    int n2 = b.length;
    if (n1 > n2) return findMedianSortedArrays(b, a);
    int low = 0, high = n1;
    int left = (n1 + n2 + 1) / 2;
    int n = n1 + n2;
    while (low <= high) {
        int mid1 = (low + high) >> 1;
        int mid2 = left - mid1;
        int l1 = (mid1 == 0) ? Integer.MIN_VALUE : a[mid1 - 1];
        int l2 = (mid2 == 0) ? Integer.MIN_VALUE : b[mid2 - 1];
        int r1 = (mid1 == n1) ? Integer.MAX_VALUE : a[mid1];
        int r2 = (mid2 == n2) ? Integer.MAX_VALUE : b[mid2];
        if (l1 <= r2 && l2 <= r1) {
            if (n % 2 == 1) return Math.max(l1, l2);
            return ((double) (Math.max(l1, l2) + Math.min(r1, r2))) / 2.0;
        } else if (l1 > r2) {
            high = mid1 - 1;
        } else {
            low = mid1 + 1;
        }
    }
    return 0.0;
}
}
```

Kth element of Two Sorted Arrays

Binary Search | Array

Problem:

- Given two **sorted arrays** a[] and b[].
- Find the element that would appear at the **kth position** in the merged sorted array.
- Must be efficient (avoid full merge).

Example:

Input: [2,3,6,7,9], [1,4,8,10], k = 5, Output: 6
Input: [1,4,8,10,12], [5,7,11,15,17], k = 6, Output: 10

Approach:

- Always binary search on **smaller array**.
- Choose partition such that:
  - Total elements in left half = k.
- Define:
  - l1, l2 → left elements
  - r1, r2 → right elements
- If l1 ≤ r2 and l2 ≤ r1:
  - kth element = max(l1, l2).
- Else adjust partition accordingly.

Complexity:

- **Time:** O(log(min(m, n)))
- **Space:** O(1)

Takeaways:

- Extension of **Median of Two Sorted Arrays**
- Key idea: **balanced partition of size k**
- Always search on smaller array

Dry Run

```java
class Solution {
    public int kthElement(int a[], int b[], int k) {
        int m = a.length;
        int n = b.length;
        if (m > n) {
            return kthElement(b, a, k);
        }
        int left = k;
        int low = Math.max(0, k - n), high = Math.min(k, m);
        while (low <= high) {
            int mid1 = (low + high) >> 1;
            int mid2 = left - mid1;
            int l1 = (mid1 > 0) ? a[mid1 - 1] : Integer.MIN_VALUE;
            int l2 = (mid2 > 0) ? b[mid2 - 1] : Integer.MIN_VALUE;
            int r1 = (mid1 < m) ? a[mid1] : Integer.MAX_VALUE;
            int r2 = (mid2 < n) ? b[mid2] : Integer.MAX_VALUE;
            if (l1 <= r2 && l2 <= r1) {
                return Math.max(l1, l2);
            } else if (l1 > r2) {
                high = mid1 - 1;
            } else {
                low = mid1 + 1;
            }
        }
        return -1;
    }
}
```

Row with max 1s

Binary Search | Array

Problem:

- Given a **2D binary matrix** (rows sorted in non-decreasing order).
- Find the **index of the first row** with the maximum number of 1s.
- Return -1 if no row contains 1.

Example:

Input: [[0,1,1,1], [0,0,1,1], [1,1,1,1], [0,0,0,0]], Output: 2
Input: [[0,0], [1,1]], Output: 1
Input: [[0,0], [0,0]], Output: -1

Approach:

- Traverse each row.
- Count number of 1s.
- Track row with maximum count.

Complexity:

- **Time:** O(n * m)
- **Space:** O(1)

```java
class Solution {
    public int rowWithMax1s(int arr[][]) {
        int m = arr.length;
        int n = arr[0].length;
        int maxOnes = 0;
        int ind = -1;
        for(int i = 0; i < m; i++) {
            int oneCnt = 0;
            for(int j = 0; j < n; j++) {
                oneCnt += arr[i][j];
            }
            if(oneCnt > maxOnes){
                maxOnes = oneCnt ;
                ind = i;
            }
        }
        return ind;
    }
}
```

Row with max 1s

Binary Search | Array

Approach:

- Since rows are sorted:
  - ind **first occurrence of 1** using lower bound.
- Number of 1s = m - firstOneIndex.
- Track row with highest count.

Complexity:

- **Time:** O(n log m)
- **Space:** O(1)

Takeaways:

- Sorted row enables **binary search**
- Lower bound gives first 1
- Efficient improvement from O(nm) → O(n log m)
- Classic **matrix + binary search** pattern

Dry Run

```java
class Solution {
    public int rowWithMax1s(int arr[][]) {
        int n = arr.length;
        int m = arr[0].length;
        int cnt_max = 0;
        int index = -1;
        for (int i = 0; i < n; i++) {
            int firstOneIndex = lowerBound(arr[i], m, 1);
            int cnt_ones = m - firstOneIndex;
            if (cnt_ones > cnt_max) {
                cnt_max = cnt_ones;
                index = i;
            }
        }
        return index;
    }
    public int lowerBound(int[] row, int cols, int x) {
        int low = 0, high = cols - 1;
        int ans = cols;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (row[mid] >= x) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return ans;
    }
}
```

74. Search a 2D Matrix

Binary Search | Array

Problem:

- Given an m × n matrix:
  - Each row is sorted.
  - First element of each row > last element of previous row.
- Determine if target exists.
- Track row with highest count.

Example:

Input: [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
Input: [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Approach:

- For each row:
  - Check if target lies within row range.
  - Apply binary search on that row.

Complexity:

- **Time:** O(n + log m) *(or roughly O(n log m))*
- **Space:** O(1)

```java
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int n = matrix.length, m = matrix[0].length;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if (matrix[i][0] <= target && target <= matrix[i][m - 1]) {
                    return bs(matrix[i], target);
                }
            }
        }
        return false;
    }
    public boolean bs(int[] row, int target) {
        int low = 0, high = row.length -1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(row[mid] == target) return true;
            else if(row[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return false;
    }
}
```

#pattern

74. Search a 2D Matrix

Binary Search | Array

Approach:

- Treat matrix as a **1D sorted array**
- Apply binary search from 0 to m*n - 1
- Convert index:
  - row = mid / m
  - col = mid % m

Complexity:

- **Time:** O(log(m × n))
- **Space:** O(1)

Takeaways:

- Matrix behaves like a **sorted 1D array**
- Index mapping trick is key insight
- Clean application of binary search
- Important **2D search pattern**

```java
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int n = matrix.length, m = matrix[0].length;
        int low = 0, high = m * n -1;
        while(low <= high) {
            int mid = (low + high) / 2;
            int row = mid / m;
            int col = mid % m;
            if(matrix[row][col] == target) return true;
            else if(matrix[row][col] < target) low = mid + 1;
            else high = mid - 1;
        }
        return false;
    }
}
```

#Pro Tip: Keep this row & col conversion from 2d to 1d in mind!

240. Search a 2D Matrix II

Binary Search | Array

Problem:

- Given an m × n matrix:
  - Each row is sorted (left → right).
  - Each column is sorted (top → bottom).
- Return true if target exists, else false.

Example:

Input: matrix = [[1,4,7,11,15], [2,5,8,12,19], [3,6,9,16,22], [10,13,14,17,24], [18,21,23,26,30]], target = 5
Output: true

Input: Same matrix, target = 20
Output: false

Approach:

- Apply binary search on every row.
- If found in any row → return true.

Complexity:

- **Time:** O(n + log m)
- **Space:** O(1)

```java
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        for(int i = 0; i < matrix.length; i++) {
            if (bs(matrix[i], target)) return true;
        }
        return false;
    }
    public boolean bs(int[] row, int target) {
        int low = 0, high = row.length - 1;
        while(low <= high) {
            int mid = (low + high) / 2;
            if(row[mid] == target) return true;
            else if(row[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return false;
    }
}
```

240. Search a 2D Matrix II

Binary Search | Array

Approach:

- Start from **top-right corner**.
- If current value > target → move left.
- If current value < target → move down.
- Eliminate one row or column at each step.

Complexity:

- **Time:** O(n + m)
- **Space:** O(1)

Takeaways:

- Not fully sorted like previous matrix problem.
- Top-right start gives directional advantage.
- Eliminate row/column each step.
- Classic **2D matrix traversal pattern**.

```java
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int row = 0, col = matrix[0].length - 1;
        while(row < matrix.length && col >= 0) {
            if(matrix[row][col] == target) return true;
            else if(matrix[row][col] < target) row++;
            else col--;
        }
        return false;
    }
}
```

1901. Find a Peak Element II

Binary Search | Array

Problem:

- Given an m × n matrix.
- A peak is strictly greater than its **top, bottom, left, right** neighbors.
- Adjacent cells are never equal.
- Return index [row, col] of **any peak**.
- Assume the entire matrix is surrounded by an **outer perimeter** with the value -1 in each cell.

Example:

Input: [[1,4],[3,2]],
Output: [0,1] *(or [1,0])*

-1	-1	-1	-1
-1	1	4	-1
-1	3	2	-1
-1	-1	-1	-1

Input: [[10,20,15],[21,30,14],[7,16,32]],
Output: [1,1] *(or [2,2])*

-1	-1	-1	-1	-1
-1	10	20	15	-1
-1	21	30	14	-1
-1	7	16	32	-1
-1	-1	-1	-1	-1

1901. Find a Peak Element II

Binary Search | Array

Approach:

- Apply binary search on **columns**.
- For mid column:
  - Find row index of **maximum element** in that column.
- Compare with left and right neighbors:
  - If greater than both → peak found.
  - If left is greater → move left.
  - Else → move right.
- Eliminates half of columns each step.

Complexity:

- **Time:** O(m log n)
- **Space:** O(1)

Takeaways:

- Extension of **1D peak finding**
- Always move toward the **larger neighbor**
- Use column max to guarantee vertical dominance
- Classic **2D binary search pattern**

```java
class Solution {
    public int[] findPeakGrid(int[][] arr) {
        int n = arr.length;
        int m = arr[0].length;
        int low = 0;
        int high = m - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            int row = maxElement(arr, mid);
            int left = mid - 1 >= 0 ? arr[row][mid - 1] : Integer.MIN_VALUE;
            int right = mid + 1 < m ? arr[row][mid + 1] : Integer.MIN_VALUE;
            if (arr[row][mid] > left && arr[row][mid] > right) {
                return new int[] { row, mid };
            } else if (left > arr[row][mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return new int[] { -1, -1 };
    }
    public int maxElement(int[][] arr, int col) {
        int n = arr.length;
        int max = Integer.MIN_VALUE;
        int index = -1;
        for (int i = 0; i < n; i++) {
            if (arr[i][col] > max) {
                max = arr[i][col];
                index = i;
            }
        }
        return index;
    }
}
```

Median in a row-wise sorted Matrix

Binary Search | Array

Problem:

- Given a **row-wise sorted matrix** of size n × m.
- n and m are always **odd**.
- Return the **median** of all elements.

Example:

Input: matrix = [[1,3,5],[2,6,9],[3,6,9]], Output: 5
Input: [[2,4,9],[3,6,7],[4,7,10]], Output: 6
Input: [[3],[4],[8]], Output: 4

Approach:

- Store all elements in a list.
- Sort the list.
- Return middle element.

Complexity:

- **Time:** O(nm log(nm))
- **Space:** O(nm)

```java
class Solution {
    public int median(int[][] matrix) {
        List<Integer> elements = new ArrayList<>();
        for (int[] row : matrix) {
            for (int val : row) {
                elements.add(val);
            }
        }
        Collections.sort(elements);
        int n = elements.size();
        return elements.get(n / 2);
    }
}
```

Median in a row-wise sorted Matrix

Binary Search | Array

Approach:

- Median position = (n*m + 1) / 2.
- Search between:
  - Minimum of first elements.
  - Maximum of last elements.
- For each mid value:
  - Count elements ≤ mid using binary search per row.
- If count < required → increase mid.
- Else → reduce search space.
- Final low is median.

Complexity:

- **Time:** O(n log m × log(range))
- **Space:** O(1)

Takeaways:

- Not fully sorted → can't flatten directly
- Use **binary search on value space**
- Combine row-wise binary search with global counting
- Classic **matrix + answer-based search**

```java
class Solution {
    public int median(int[][] matrix) {
        int rows = matrix.length;
        int cols = matrix[0].length;
        int low = matrix[0][0];
        int high = matrix[0][cols - 1];
        for (int i = 1; i < rows; i++) {
            low = Math.min(low, matrix[i][0]);
            high = Math.max(high, matrix[i][cols - 1]);
        }
        while (low < high) {
            int mid = (low + high) / 2;
            int count = 0;
            for (int i = 0; i < rows; i++) {
                count += countLessEqual(matrix[i], mid);
            }
            if (count < (rows * cols + 1) / 2)
                low = mid + 1;
            else
                high = mid;
        }
        return low;
    }
        int countLessEqual(int[] row, int mid) {
        int low = 0, high = row.length;
        while (low < high) {
            int m = (low + high) / 2;
            if (row[m] <= mid) low = m + 1;
            else high = m;
        }
        return low;
    }
}
```

1021. Remove Outermost Parentheses

Stack | String

Problem:

- Given a **valid parentheses string** s.
- Split it into its **primitive components**.
- Remove the **outermost parentheses** from each primitive.
- Return the final modified string.

Example:

Input: "(()())(())", Output: "()()()“
Input: "(()())(())(()(()))", Output: "()()()()(())"
Input: "()()", Output: ""

Approach:

- Maintain a depth counter.
- For '(':
  - Add to result only if depth > 0.
  - Then increase depth.
- For ')':
  - Decrease depth first.
  - Add to result only if depth > 0.
- This skips the outermost layer of each primitive.

Complexity:

- **Time:** O(n)
- **Space:** O(n)

```java
class Solution {
    public String removeOuterParentheses(String s) {
        StringBuilder result = new StringBuilder();
        int depth = 0;
        for (char ch : s.toCharArray()) {
            if (ch == '(') {
                if (depth > 0) {
                    result.append(ch);
                }
                depth++;
            }
            else {
                depth--;
                if (depth > 0) {
                    result.append(ch);
                }
            }
        }
        return result.toString();
    }
}
```

Takeaways:

- Primitive structure identified via **depth = 0 reset**
- Simple counter replaces stack
- Classic **parentheses parsing pattern**
- Very common in string + stack problems

151. Reverse Words in a String

Two Pointers | String

Problem:

- Given a string s.
- Reverse the **order of words**.
- Remove:
  - Leading spaces, trailing spaces, extra spaces between words
- Return words separated by a **single space**.

Example:

Input: "the sky is blue", Output: "blue is sky the"
Input: " hello world ", Output: "world hello"
Input: “a good example", Output: “example good a"

Approach:

- Traverse string from **right to left**.
- Skip trailing spaces.
- Identify word boundaries.
- Extract word using substring.
- Append to result with a single space.

Complexity:

- **Time:** O(n)
- **Space:** O(n)

```java
class Solution {
    public String reverseWords(String s) {
        StringBuilder ans = new StringBuilder();
        int ind = s.length() - 1;
        while(ind >= 0) {
            while(ind >= 0 && s.charAt(ind) == ' ') ind--;
            if(ind < 0) break;
            int wrdEndInd = ind;
            while(ind >= 0 && s.charAt(ind) != ' ') ind--;
            String word = s.substring(ind + 1, wrdEndInd + 1);
            if(ans.length() > 0) ans.append(' ');
            ans.append(word);
        }
        return ans.toString();
    }
}
```

Takeaways:

- No need to split → avoids extra space overhead
- Right-to-left traversal simplifies trimming
- Carefully manage spaces
- Common **string parsing pattern**

1903. Largest Odd Number in String

Greedy | String

Problem:

- Given a numeric string num.
- Find the **largest-valued odd integer substring**.
- Return empty string "" if no odd substring exists.

Example:

Input: “52", Output: “5"
Input: “4206", Output: ""
Input: "35427", Output: "35427"

Approach:

- Traverse from **right to left**.
- Check each digit:
  - If digit is **odd**, return substring from 0 to that index.
- First odd digit from right gives **largest possible odd substring**.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public String largestOddNumber(String num) {
        for(int i = num.length() - 1; i >= 0; i--) {
            int curDig = num.charAt(i) - '0';
            if(curDig % 2 != 0) {
                return num.substring(0, i + 1);
            }
        }
        return "";
    }
}
```

Takeaways:

- Largest substring must start at index 0
- Only need to check **last odd digit**
- Simple greedy observation
- Efficient string handling problem

14. Longest Common Prefix

Array String

Problem:

- Given an array of strings strs.
- Find the **longest common prefix** among all strings.
- Return "" if none exists.

Example:

Input: ["flower","flow","flight"], Output: "fl"
Input: ["dog","racecar","car"], Output: ""

Approach:

- Sort the array lexicographically.
- Compare only:
  - First String
  - Last Sting
- Common prefix of these two gives answer.
- Why? Sorting pushes most different strings to extremes.

```java
class Solution {
    public String longestCommonPrefix(String[] strs) {
        StringBuilder ans = new StringBuilder();
        Arrays.sort(strs);
        String first = strs[0];
        String last = strs[strs.length - 1];
        for(int i = 0; i < Math.min(first.length(), last.length()); i++) {
            if(first.charAt(i) != last.charAt(i)){
                return ans.toString();
            } else ans.append(first.charAt(i));
        }
        return ans.toString();
    }
}
```

Complexity:

- **Time:** O(n log n + m) (*sorting + character comparison*)
- **Space: Space:** O(1) (*ignoring sort overhead*)

(Where n = number of strings, m = length of smallest string)

Takeaways:

- Key idea: only compare **first and last** after sorting
- Sorting simplifies multi-string comparison
- Elegant and concise solution
- Classic string interview problem

205. Isomorphic Strings

String

Problem:

- Given two strings s and t.
- Determine if s can be transformed into t using a **one-to-one character mapping**.
- Rules:
  - Each character in s maps to one character in t.
  - No two characters map to the same character.
  - Order must be preserved.

Example:

Input: "egg", "add" , Output: true
Input: "f11", "b23", Output: false
Input: "paper", "title", Output: true

Approach:

- Use two arrays of size 256:
  - m1 → map for s
  - m2 → map for t
- Traverse both strings:
  - If previously mapped positions differ → return false.
  - Update both mappings with i + 1.
- i + 1 avoids confusion with default 0.

```java
class Solution {
    public boolean isIsomorphic(String s, String t) {
        int[] m1 = new int[256], m2 = new int[256];
        int n = s.length();
        for (int i = 0; i < n; ++i) {
            if (m1[s.charAt(i)] != m2[t.charAt(i)])
                return false;
            m1[s.charAt(i)] = i + 1;
            m2[t.charAt(i)] = i + 1;
        }
        return true;
    }
}
```

Complexity:

- **Time:** O(n)
- **Space:** O(1) (constant size arrays)

205. Isomorphic Strings

String

Takeaways:

- Need **bidirectional mapping**
- Last seen index ensures consistency
- No HashMap needed → array works faster
- Classic **mapping / pattern matching problem**

#Pro Tip: If you need to touch every character, use an array.

```java
class Solution {
    public boolean isIsomorphic(String ss, String tt) {
        int[] ms = new int[256];
        int[] mt = new int[256];
        char[] s = ss.toCharArray();
        char[] t = tt.toCharArray();
        for (int i = 0; i < s.length; i++) {
            if (ms[s[i]] != mt[t[i]]) return false;
            ms[s[i]] = i+1;
            mt[t[i]] = i+1;
        }
        return true;
    }
}
```

796. Rotate String

String

Problem:

- Given two strings s and goal.
- Check if goal can be obtained by **rotating** s any number of times.
- A rotation moves first character to the end.

Example:

Input: "abcde", "cdeab", Output: true
Input: "abcde", "abced", Output: false

Approach:

- If lengths differ → return false.
- Concatenate string with itself: s + s.
- If goal is a substring of (s + s) → return true.
- Why it worls:
  - All possible rotations of s exist inside s + s.

```java
class Solution {
    public boolean rotateString(String s, String goal) {
        if(s.length() != goal.length()) return false;
        String doubledS = s + s;
        return doubledS.contains(goal);
    }
}
```

Complexity:

- **Time:** O(n)
- **Space: Space:** O(n)

Takeaways:

- Smart use of string concatenation
- Classic rotation detection trick
- No need to simulate shifts
- Very common interview pattern

242. Valid Anagram

Sorting | String

Problem:

- Given two strings s and t.
- Return true if t is an **anagram** of s.
- Anagram → same characters with same frequency.

Example:

Input: "anagram", "nagaram", Output: true
Input: "rat", "car", Output: false

Approach:

- Convert both strings to char arrays.
- Sort both arrays.
- Compare arrays.

Complexity:

- **Time:** O(n log n) ➔ Coz of sorting
- **Space: Space:** O(n + m)

```java
class Solution {
    public boolean isAnagram(String s, String t) {
        char[] a = s.toCharArray();
        char[] b = t.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        return Arrays.equals(a,b);
    }
}
```

Can be improved

#pattern

242. Valid Anagram

Hashing | String

Approach:

- Basic checks :: If Input lengths differ → return false.
- Use frequency array of size 26
- Increment for s, decrement for t.
- If all values are 0 → valid anagram.

Complexity:

- **Time:** O(n)
- **Space:** O(26)

Takeaways:

- Sorting is simple but slower.
- Frequency array gives linear time.
- Very common string frequency pattern.
- Foundation for many problems (Group Anagrams, Sliding Window)

```java
class Solution {
    public boolean isAnagram(String str1, String str2) {
        if (str1.length() != str2.length())
            return false;
        int[] freq = new int[26];
        for (int i = 0; i < str1.length(); i++) {
            freq[str1.charAt(i) - 'a']++;
        }
        for (int i = 0; i < str2.length(); i++) {
            freq[str2.charAt(i) - 'a']--;
        }
        for (int i = 0; i < 26; i++) {
            if (freq[i] != 0)
                return false;
        }
        return true;
    }
}
```

#pattern

242. Valid Anagram

Hashing | String

Follow up: What if the inputs contain special charecters, Uppercase alphabate etc...
How would you adapt your solution to such a case?

Approach:
The concepts is same, just instead of 26 size of array we are using hashMap
Take HashMap with Key → Character & Value → count

Complexity:
- **Time:** O(n)
- **Space:** O(n)

```java
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length())
            return false;
        Map<Character, Integer> counts = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            counts.merge(s.charAt(i), 1, Integer::sum);
        }
        for (int i = 0; i < t.length(); i++) {
            counts.merge(t.charAt(i), -1, Integer::sum);
        }
        for(int count : counts.values()) {
            if(count != 0) return false;
        }
        return true;
    }
}
```

See how we can iterate hashMap w.r.t. value directly

#pattern

49. Group Anagrams

Hashing | String

Example:
Input: strs = ["eat","tea","tan","ate","nat","bat"]
Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Input: strs = [""] Output: [[""]]

Input: strs = ["a"] Output: [["a"]]

Approach:
To solve this problem, we can use a hash map to group strings that are anagrams. The key idea is to sort each string and use the sorted string as the key in the hash map.

Step:

- **Initialize a Hash Map**: Create a hash map where key is the sorted string and value is a list of strings that are anagrams.
- **Sort Each String**: Iterate through the input array, sort each string, and use the sorted string as the key to group the original strings in the hash map.
- **Return the Result**: Collect all the values from the hash map and return them as the result.

```java
class Solution {
    // ["eat","tea","tan","ate","nat","bat"]
    public List<List<String>> groupAnagrams(String[] strs) {
        if(strs.length == 0 || strs == null) return new ArrayList<>();
        Map<String, List<String>> angrmGrp = new HashMap<>();
        for(String s : strs) {
            //  "eat"
            char[] charArr = s.toCharArray();
            // ['e', 'a', 't']
            Arrays.sort(charArr);
            // ['a', 'e', 't']
            String key = new String(charArr);
            // checking if this key is there or not
            if(!angrmGrp.containsKey(key)) {
                // if not, then creating a new one
                angrmGrp.put(key, new ArrayList<>());
            }
            // else, adding the string in exisitng one
            angrmGrp.get(key).add(s);
        }
        return new ArrayList<>(angrmGrp.values());
    }
}
```

See how we can initialize a String from charArr

See how we are getting the key and adding value to that key

Complexity Analysis

- **Time Complexity:** $\mathcal{O}(N \cdot K \log K)$, where N is the number of strings (`strs.length`) and K is the maximum length of a string (`strs[i].length`). Sorting each string of length K takes $\mathcal{O}(K \log K)$ time, performed for all N strings.
- **Space Complexity:** $\mathcal{O}(N \cdot K)$ to store the grouped strings and keys in the `HashMap` .

#pattern

49. Group Anagrams

Hashing | String

Approach:

Instead of sorting each string, count the frequency of each character (since strings contain only lowercase English letters). Constructing a frequency key takes linear time relative to the string length.

Complexity:

- **Time:** $\mathcal{O}(N \cdot K)$
- **Space:** $\mathcal{O}(N \cdot K)$

When to use which approach:

- **Lowercase English letters (`'a'` to `'z'`):** Frequency Counting Array (`int[26]`) is optimal ($\mathcal{O}(N \cdot K)$ time).
- **Arbitrary ASCII / Full Unicode / Special Characters:** Sorting (`Arrays.sort(charArray)`) is cleaner and more space-efficient in practice, making it the preferred solution.

Dry Run

```java
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        if (strs == null || strs.length == 0) return new ArrayList<>();
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            int[] count = new int[26];
            for (char ch : s.toCharArray()) {
                count[ch - 'a']++;
            }
            String key = Arrays.toString(count);
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

This line condenses a multi-step HashMap check-and-insert pattern into a single atomic operation:

Step-by-Step Breakdown

1. `map.computeIfAbsent(key, ...)`: Checks if the given `key` already exists in `map`.
2. `k -> new ArrayList<>()`: If `key` is **not present** (or maps to `null`), this lambda expression runs, creates a brand new `ArrayList`, and puts it into the map paired with `key`. If `key` **already exists**, this part is skipped entirely.
3. **Return Value:** `computeIfAbsent` always returns the `List<String>` associated with `key` (whether it was just created or previously existed).
4. `.add(s)`: Immediately appends string `s` to that retrieved list.

Note: While frequency counting reduces the theoretical time complexity from $\mathcal{O}(N \cdot K \log K)$ to $\mathcal{O}(N \cdot K)$, sorting is often faster for small string lengths ($K \leq 100$) due to low overhead.

#pattern

125. Valid Palindrome

Two Pointers | String

Thought

It's a no brainer question!

The main thing is we should be aware of all the methods of Character, String/StringBuilder to manipulate the given input

Check these out::

- Most useful Character comparison method
- All About String method
- All StringBuilder methods

Brute Approach:

Once we clean the data, saving the clnStr
simply keep a track of left and right character
If it's mismatch return false
At the end return true

Optimal Approach:

No need to maintain clnStr :: Do everything on the Go!
This will reduce the Space complexity to O(n) → O(1)

```java
class Solution {
    public boolean isPalindrome(String s) {
        StringBuilder clnStr = new StringBuilder();
        for (char ch : s.toCharArray()) {
            if (Character.isLetterOrDigit(ch)) {
                clnStr.append(Character.toLowerCase(ch));
            }
        }
        int l = 0;
        int r = clnStr.length() - 1;
        while (l < r) {
            if (clnStr.charAt(l) != clnStr.charAt(r))
                return false;
            l++;
            r--;
        }
        return true;
    }
}
```

```java
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while(l < r) {
            while(l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
            while(l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
            if(Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
            return false;
            l++;
            r--;
        }
        return true;
    }
}
```

#pattern

13. Roman to Integer

Math | String

Problem:

- Given a Roman numeral string s.
- Convert it to its **integer value**.
- Roman numerals include special **subtraction cases** (IV, IX, XL, XC, CD, CM).

Example:

Input: “III", Output: 3
Input: "LVIII", Output: 58
Input: "MCMXCIV", Output: 1994

Approach:

- Traverse string from **right to left**.
- Convert Roman symbol to its value.
- If current value < previous → **subtract**.
- Otherwise → **add**.
- Keep track of previous value.

Complexity:

- **Time:** O(n) — single traversal
- **Space:** O(1) — constant variables

Takeaways:

- Reverse traversal simplifies subtraction logic.
- Smaller-before-larger means **subtraction case**.
- Common pattern for **symbol-to-value conversion**.

```java
class Solution {
    public int romanToInt(String s) {
        int num = 0, result = 0, prev = 0;
        for (int i = s.length() - 1; i >= 0; i--) {
            switch (s.charAt(i)) {
                case 'I':
                    num = 1;
                    break;
                case 'V':
                    num = 5;
                    break;
                case 'X':
                    num = 10;
                    break;
                case 'L':
                    num = 50;
                    break;
                case 'C':
                    num = 100;
                    break;
                case 'D':
                    num = 500;
                    break;
                case 'M':
                    num = 1000;
                    break;
            }
            if (num < prev) {
                result -= num;
            } else {
                result += num;
            }
            prev = num;
        }
        return result;

    }
}
```

3. Longest Substring Without Repeating Characters

Slliding Window | String

Thought:

- Maintain a window which contain only unique charecters
- Keep track of current window length
- If duplicate charecter found, remove charecter from left until it's valid again
- check if current length (right - left + 1) is greater than maxLen, if so update it

Complexity:

- **Time:** O(n)
- **Space:** O(n)

```java
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Set<Character> set = new HashSet<>();
        int maxLen = 0;
        int left = 0;
        for(int right = 0; right < s.length(); right++) {
            while(set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

#pattern

424. Longest Repeating Character Replacement

Slliding Window | String

Thought:

- Let's create all possible substring, also a maxLen to save the longest string
- Keep a track of character count into an array of length 26
- Also, taking two variable for tracking maxFreq & change
  - Change means how many replachment should be done, and if we think carefully, we should always swap the remaining charecters other than the max one.
- Next, while traversing the substring keep updating the char count in array
- And calculate the latest change as well
- Now, If the change is withing k then check if maxLen should be updated with current substring or not
- At the end of the day return maxLen

```java
class Solution {
    public int characterReplacement(String s, int k) {
        int maxLen = 0;
        for(int i = 0; i < s.length(); i++) {
            int[] hs = new int[26];
            int maxFrq = 0, chngs = 0;
            for(int j = i; j < s.length(); j++) {
                hs[s.charAt(j) - 'A']++;
                maxFrq = Math.max(maxFrq, hs[s.charAt(j) - 'A']);
                chngs = (j - i + 1) - maxFrq;
                if(chngs <= k) {
                    maxLen = Math.max(maxLen, j - i + 1);
                } else break;
            }
        }
        return maxLen;
    }
}
```

Complexity:

- **Time:** $O(n^2)$
- **Space:** O(z) :: where z = 26

IT HAS ONE OPTIMAL SLIDING WINDOW APPROACH WHICH I NEED TO UNDERSTAND :p

#pattern

567. Permutation in String

Slliding Window | String

Thought:

- If you are thinking of keeping all the possible permutation string in some data structure and itering one by one and check against s2 i.e. *s2.contains(s1_permutation)*;
- Let me tell you :: If s1 has length n, there are n! possible permutations :p
- Think this problem in this way :: *Does s2 contain any contiguous substring of length s1.length() that has the exact same character frequencies as s1?*
- Maintain two frequency arrays of size 26 (for lowercase English letters).
- Count frequencies of all characters in s1 and the first window of s2
- Slide the window across s2 one character at a time:
  - Add the incoming character count.
  - Remove the outgoing character count.
  - Compare the two frequency arrays. If they match, return true.

Complexity:

- **Time:** O(n)
- **Space:** O(1)

```java
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int s1Len = s1.length();
        int s2Len = s2.length();
        if (s1Len > s2Len) return false;
        int[] s1Hs = new int[26];
        int[] s2Hs = new int[26];
        // Populate initial window (indices 0 to s1Len - 1)
        for (int i = 0; i < s1Len; i++) {
            s1Hs[s1.charAt(i) - 'a']++;
            s2Hs[s2.charAt(i) - 'a']++;
        }
        int left = 0;
        int right = s1Len; // Points to the next character to enter the window
        while (right < s2Len) {
            if (matches(s1Hs, s2Hs)) return true;
            // Expand right, shrink left
            s2Hs[s2.charAt(right) - 'a']++;
            s2Hs[s2.charAt(left) - 'a']--;

            left++;
            right++;
        }
        return matches(s1Hs, s2Hs);
    }
    private boolean matches(int[] s1Hs, int[] s2Hs) {
        for (int i = 0; i < 26; i++) {
            if (s1Hs[i] != s2Hs[i]) return false;
        }
        return true;
    }
}
```

#pattern

239. Sliding Window Maximum

Slliding Window | Array

Thought:

- Simply making an ans array
- Iterate in the given array, while keeping left and right track
- Calling a method, which calculate maxVal of of the given range
- But this approach is very inefficient an optimal approach include Dequeue method, which I will cover later :p

Complexity:

- **Time:** O(n^2)
- **Space:** O(1)

```java
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] ans = new int[n - k + 1];
        int left = 0;
        int ind = 0;
        for (int right = k - 1; right < n; right++) {
            ans[ind] = maxVal(nums, left, right);
            left++;
            ind++;
        }
        return ans;
    }

    public int maxVal(int[] arr, int l, int r) {
        int max = Integer.MIN_VALUE;
        for (int i = l; i <= r; i++) {
            max = Math.max(max, arr[i]);
        }
        return max;
    }
}
```

#pattern

344. Reverse String

Recurssion | String

Example:

Input: s = ["h","e","l","l","o"]

Output: ["o","l","l","e","h"]

Every recursive function needs three ingredients:

The Base Case: *When do we stop?*

The Work (Current Step): *What do we do right now?*

The Recursive Call: *How do we move forward?*

Recursive Approach:

- A recursive solution here is conceptually a two-pointer approach disguised!
- Instead of using a while loop to move the pointers inward, we use **function calls and the call stack** to handle the state.

```
34      public static void revStr(char[] str) {  1 usage
35          revHelper(str, l: 0, r: str.length - 1);
36      }
37      public static void revHelper(char[] str, int l, int r) {
38          // base condition
39          if(l >= r) return;
40          // the tasks
41          char temp = str[l];
42          str[l] = str[r];
43          str[r] = temp;
44          // recursive call
45          revHelper(str, ++l, --r);
46      }
47  }
```

Look here we did *++l* and not *l++*

THINK WHY!

Iterative to Recursive

```
1  class Solution {
2      public void reverseString(char[] s) {
3          int l = 0, r = s.length - 1;
4          while(l <= r) {
5              swap(s, l, r);
6              l++;
7              r--;
8          }
9      }
10
11     public void swap(char[] s, int i, int j) {
12         char temp = s[i];
13         s[i] = s[j];
14         s[j] = temp;
15     }
16 }
```

#pattern

Complexity:

- **Time:** O(n)
- **Space:** O(n) — Recursive Stack

Any problem that can be solved iteratively can always be solved recursively and vice versa

Is Prime

Recurssion | Math

We know what a prime number is, no need to disscuss that!
Want to disscuss :: *What happens when we make a lots of recursive calls?*

Input : 2229, Output : false Okay, it's working, That's fine!
Input : 12891, Output : ***Exception in thread "main" java.lang.StackOverflowError***

Why is this happening?

In your `divCnt` function, you are making a recursive call for **every single number** from `1` up to `num`.

- If `num` is a 4-digit number (e.g., `5,000`), your code creates roughly **5,000 nested stack frames** in memory, which Java can usually handle.
- If `num` is a 5-digit number (e.g., `50,000`), your code tries to stack **50,000 method calls** on top of each other before any of them return.

Every time a method is called, Java allocates a "stack frame" to keep track of local variables and return addresses. The Java Virtual Machine (JVM) sets a strict limit on the total memory allocated for this call stack (usually a few megabytes). Once you exceed that limit, Java throws the dreaded `java.lang.StackOverflowError`.

```
    public static boolean isPrime(int num) {  1 usage
        return (divCnt(num, i: 1) == 2);
    }

    public static int divCnt(int num, int i) {  3 usages
//      Base case
        if (i > num) return 0;
//      Task
        if(num % i == 0) {
            return 1 + divCnt(num, i: i + 1);
        }
//      Recursive call
        return divCnt(num, i: i + 1);
    }
```

Keep these small points in mind, these will help you driving the interview :p

344. Reverse String

Recurssion | String

Example:

Input: s = ["h","e","l","l","o"]

Output: ["o","l","l","e","h"]

Every recursive function needs three ingredients:

The Base Case: *When do we stop?*

The Work (Current Step): *What do we do right now?*

The Recursive Call: *How do we move forward?*

Recursive Approach:

- A recursive solution here is conceptually a two-pointer approach disguised!
- Instead of using a while loop to move the pointers inward, we use **function calls and the call stack** to handle the state.

```
34      public static void revStr(char[] str) {  1 usage
35          revHelper(str, l: 0, r: str.length - 1);
36      }
37      public static void revHelper(char[] str, int l, int r) {
38          // base condition
39          if(l >= r) return;
40          // the tasks
41          char temp = str[l];
42          str[l] = str[r];
43          str[r] = temp;
44          // recursive call
45          revHelper(str, ++l, --r);
46      }
47  }
```

Look here we did *++l* and not *l++*

THINK WHY!

Iterative to Recursive

```
1  class Solution {
2      public void reverseString(char[] s) {
3          int l = 0, r = s.length - 1;
4          while(l <= r) {
5              swap(s, l, r);
6              l++;
7              r--;
8          }
9      }
10
11     public void swap(char[] s, int i, int j) {
12         char temp = s[i];
13         s[i] = s[j];
14         s[j] = temp;
15     }
16 }
```

#pattern

Complexity:

- **Time:** O(n)
- **Space:** O(n) — Recursive Stack

Any problem that can be solved iteratively can always be solved recursively and vice versa

509. Fibonacci Number

Recurssion | Math

Fibonacci Series :: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377,...

Mathematically ::

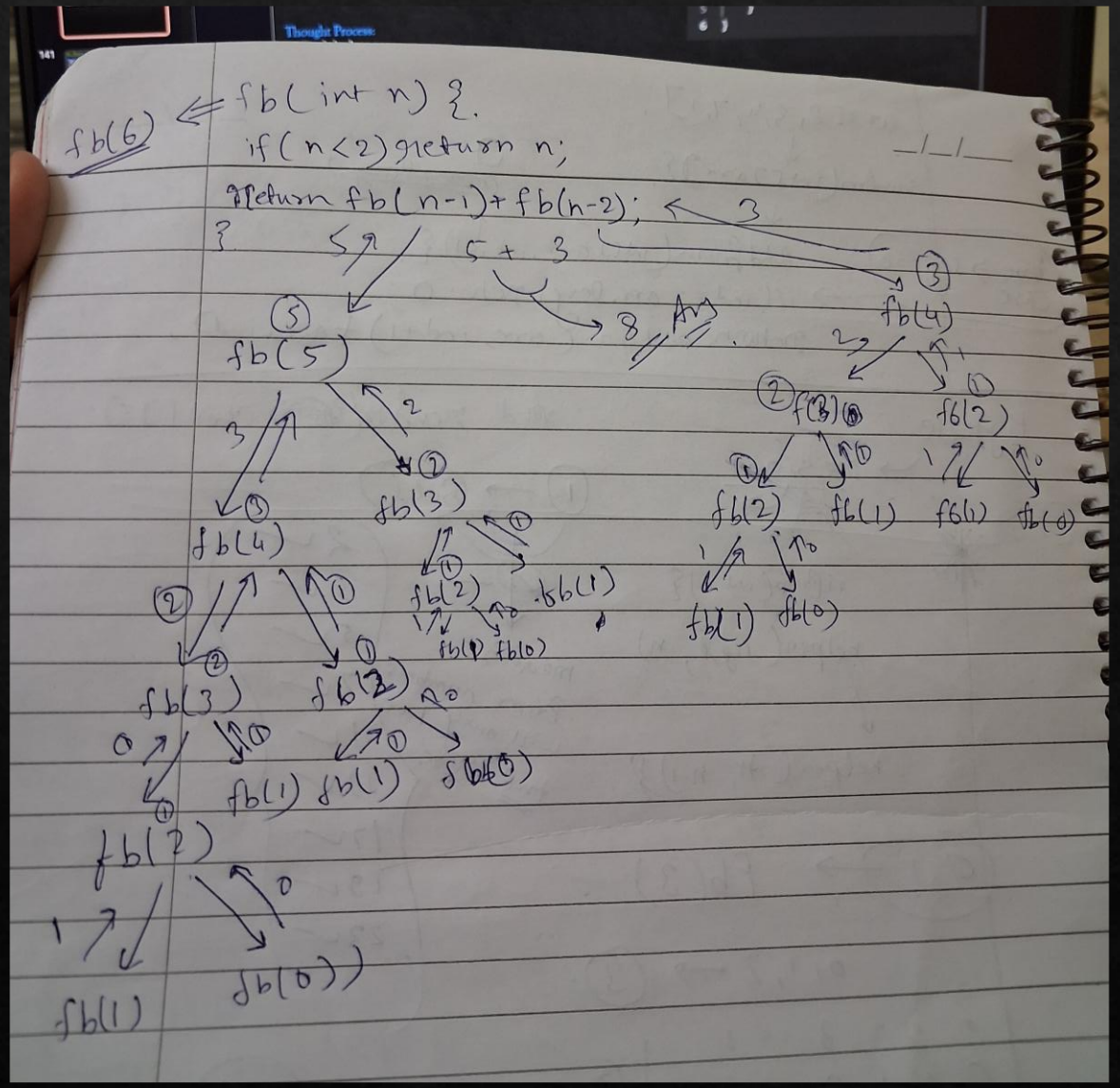
$$F(0) = 0,\ F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2),\ for\ n > 1$$

Thought Process:

- Find the base case
- Apply the recursive call
- Do a dry run using pen & paper to understand ☺

Dry Run

```
1  class Solution {
2      public int fib(int n) {
3          if(n < 2) return n;
4          return fib(n - 1) + fib(n - 2);
5      }
6  }
```



#pattern

50. Pow(x, n)

Recurssion | Math

Thought Process:

- First thing we need to figure out how to handle the –ve power
- Then we can think of Recursive approach
  - Base case is simple i.e. if power became 0 return 1
  - Here we are calculating for half of the power and then checking if
    - $x^n = (x^{n/2}) \times (x^{n/2})$ if n is even
    - $x^n = x \times (x^{n/2}) \times (x^{n/2})$ if n is odd

Constraints

- $-100.0 < x < 100.0$
- $-2^{31} <= n <= 2^{31}-1$
- n is an integer
- Either x is not zero or $n > 0$
- $-10^4 <= x^n <= 10^4$

```java
class Solution {
    public double myPow(double x, int n) {
        // long handles Integer.MIN_VALUE safely
        long N = Math.abs((long) n);
        double ans = recursion(x, N);
        // x^-n = 1 / x^n
        return n < 0 ? 1 / ans : ans;
    }
    public double recursion(double x, long n) {
        // Base case
        if (n == 0) {
            return 1;
        }
        // Reduce problem size by half
        double temp = recursion(x, n / 2);
        if (n % 2 == 0) {
            return temp * temp;
        } else {
            return x * temp * temp;
        }
    }
}
```

keeping x positive during recursion and applying the fraction at the very end to avoid that precision drift

Wrong Answer 307 / 309 testcases passed

submitted at Oct 10, 2026 23:41

Input

x =
−0.9999999968539456

n =
−1669585506

Output
191.06372

Expected
191.06370

```java
class Solution {
    public double myPow(double x, int n) {
        long longN = n;
        if (longN < 0) {
            x = 1 / x;
            longN = -longN;
        }
        return helper(x, longN);
    }

    public double helper(double x, long n) {
        // base case
        if (n == 0) return 1.0;
        double half = helper(x, n / 2);
        if (n % 2 == 0) {
            return half * half;
        } else {
            return x * half * half;
        }
    }
}
```

Inverting the base up front (x = 1/x) compounded tiny rounding errors over the recursive call

#pattern

Merge Sort

Recurssion | Sorting

Thought Process:

- Divide array into two parts
- Get both parts sorted via recursion
- Merge the sorted part

Here in this approach we are creating a copy of the subarray for recursive call

Think of doing in-place

i.e. No need to create new copy of array, use the original one and modify that only.

For in-place, we need to play with three pointer start, mid, end

Perform dry run using pen and paper to understand ☺

```java
public static int[] mergeSort(int[] arr) {
    if(arr.length == 1) return arr;
    int mid = arr.length / 2;
    // Step 2 :: Recursive call
    int[] left = mergeSort(Arrays.copyOfRange(arr, 0, mid));
    int[] right = mergeSort(Arrays.copyOfRange(arr, mid, arr.length));
    return merge(left, right);
}
// Step 3 :: Merge the sorted part
public static int[] merge(int[] first, int[] second) {
    int l = 0, r = 0, k = 0;
    int[] mix = new int[first.length + second.length];
    while(l < first.length && r < second.length) {
        if(first[l] < second[r]) {
            mix[k] = first[l];
            l++;
        } else {
            mix[k] = second[r];
            r++;
        }
        k++;
    }
    while (l < first.length) {
        mix[k] = first[l];
        k++;
        l++;
    }
    while (r < second.length) {
        mix[k] = second[r];
        k++;
        r++;
    }
    return mix;
}
```

```java
public static void mergeSortInPlace(int[] arr, int l, int r) {
    if(r - l == 1) return;
    int mid = (l + r) / 2;
    mergeSortInPlace(arr, l, mid);
    mergeSortInPlace(arr, mid, r);

    mergeInPlace(arr, l, mid, r);
}
public static void mergeInPlace(int[] arr, int s, int m, int e) {
    int l = s, r = m, k = 0;
    int[] mix = new int[e - s];
    while(l < m && r < e) {
        if(arr[l] < arr[r]) {
            mix[k] = arr[l];
            l++;
        } else {
            mix[k] = arr[r];
            r++;
        }
        k++;
    }
    // it may be possible that one of the arrays is not complete
    // copy the remaining elements
    while (l < m) {
        mix[k] = arr[l];
        k++;
        l++;
    }
    while (r < e) {
        mix[k] = arr[r];
        k++;
        r++;
    }
    for (int i = 0; i < mix.length; i++) {
        arr[s + i] = mix[i];
    }
}
```

#pattern

Quick Sort

Recurssion | Sorting

Thought Process:

- Choose any element as pivot value (P)
  - After first pass all the elements < P will be on LHS & elements > P will be at RHS of P
- i.e. P will fall automatically into it's correct position

Perform dry run using pen and paper to understand ☺

```java
static void sort(int[] arr, int low, int high) {
    if(low >= high) return;
    int start = low;
    int end = high;
    int mid = (start + end) / 2;
    int pivot = arr[mid];
    while (start <= end) {
        while (arr[start] < pivot) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
        if(start <= end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
    sort(arr, low, end);
    sort(arr, start, high);
}
```

#pattern

8. String to Integer (atoi)

Recurssion | String

Takeaways:

- Find the base case!

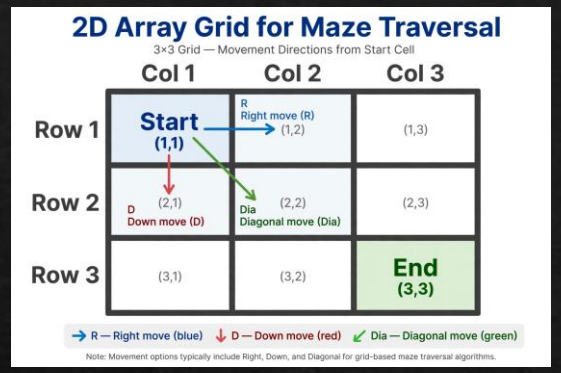
Dry Run

```java
class Solution {
    public int myAtoi(String s) {
        int i = 0;
        // skip whitespace
        while(i < s.length() && s.charAt(i) == ' ') i++;
        // handle sign
        int sign = 1;
        if(i < s.length() && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            sign = (s.charAt(i) == '-' ? -1 : 1);
            i++;
        }
        // recursive call
        return helper(s, i, 0, sign);
    }
    // recursive helper
    static int helper(String s, int i, long num, int sign) {
        // base case: end or non-digit
        if(i >= s.length() || !Character.isDigit(s.charAt(i))) {
            return (int)(sign * num);
        }
        // update num
        num = num * 10 + (s.charAt(i) - '0');
        // handeling rounding
        if(sign * num <= Integer.MIN_VALUE) return Integer.MIN_VALUE;
        if(sign * num >= Integer.MAX_VALUE) return Integer.MAX_VALUE;
        // recursive call
        return helper(s, i + 1, num, sign);
    }
```

Escape Maze

Recurssion | Sorting

Perform dry run using pen and paper to understand ☺




```java
//    Q8.3 maze with obstacle
public static void mazeWithObstacle(String p, boolean[][] maze, int r, int c) {
    if (r == maze.length - 1 && c == maze[0].length - 1) {
        System.out.println(p);
        return;
    }
    if (!maze[r][c]) return;

    if(r < maze.length - 1) {
        mazeWithObstacle(p + 'R', maze, r + 1, c);
    }
    if(c < maze[0].length - 1) {
        mazeWithObstacle(p + 'D', maze, r, c + 1);
    }
    if(r < maze.length - 1 && c < maze[0].length - 1) {
        mazeWithObstacle(p + "Dia", maze, r + 1, c + 1);
    }
```

```java
//    Q8 maze escape problem
public static int maze(int r, int c) {
    if (r == 1 || c == 1) return 1;
    int down = maze(r - 1, c);
    int right = maze(r, c - 1);
    return down + right;
}

//    Q8.1 print path
public static void mazePath(String p, int r, int c) {
    if (r == 1 && c == 1 ) {
        System.out.println(p);
        return;
    }
    if(r > 1) {
        mazePath(p + 'R', r - 1, c);
    }
    if(c > 1) {
        mazePath(p + 'D', r, c - 1);
    }
}

//    Q8.2 - diagonal travel included are returning in ArrayList
public static List<String> mazePathDiagonal(String p, int r, int c) {
    List<String> ans = new ArrayList<>();
    if (r == 1 && c == 1 ) {
        ans.add(p);
        return ans;
    }
    if(r > 1) {
        ans.addAll(mazePathDiagonal(p + 'R', r - 1, c));
    }
    if(c > 1) {
        ans.addAll(mazePathDiagonal(p + 'D', r, c - 1));
    }
    if(r > 1 && c > 1) {
        ans.addAll(mazePathDiagonal(p + "Dia", r - 1, c - 1));
    }
    return ans;
}
```

#pattern

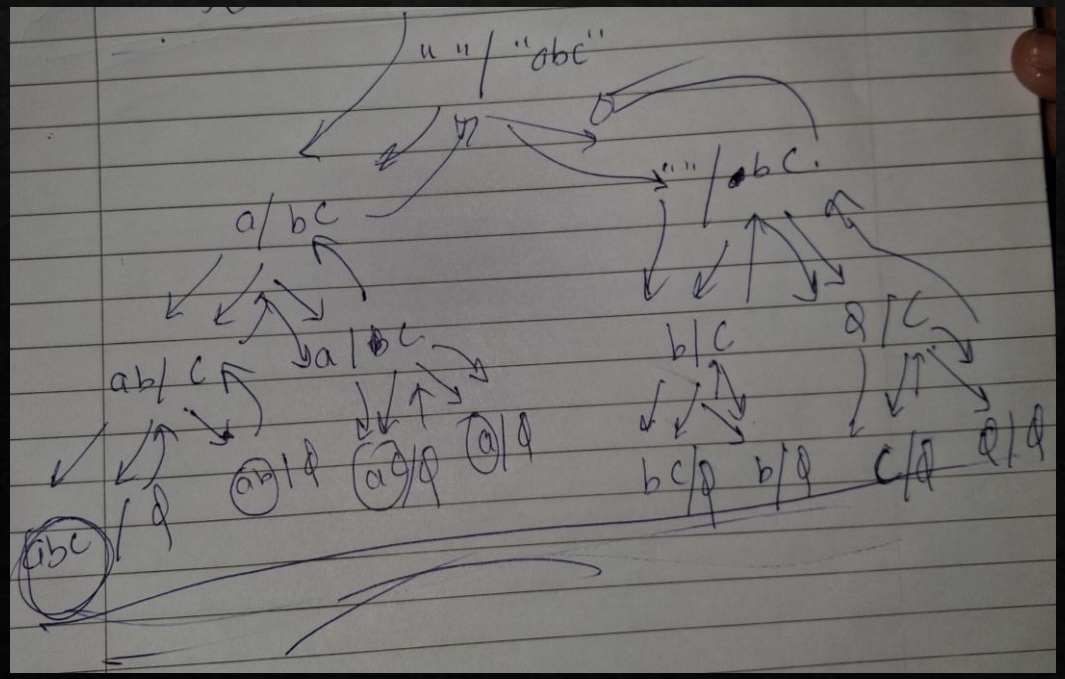
Subsequence

Recurssion | Backtracking

Input: str = abc
Output: print all subsequence i.e. *abc, ab, ac, a, bc, b, c, " "*

Thought Process

- Classic **pick/skip pattern** on strings. At every step, we will make two choices:
  - Include the current character (processed + ch).
  - Exclude the current character (processed).
- When applied to strings, this logic generates all possible subsequences.



Dry Run

```java
public static void subSeqString(String str) {
    helper("", str);
}
public static void helper(String processed, String unprocessed) {
    if (unprocessed.isEmpty()) {
        System.out.println(processed);
        return;
    }
    char ch = unprocessed.charAt(0);
    helper(processed + ch, unprocessed.substring(1));
    helper(processed, unprocessed.substring(1));
}
```

Notice that when we pass *processed + ch* into the first call, Java creates a **brand-new string**. Original *processed* variable remains unchanged. Because strings are immutable in Java, we never had to "undo" anything — because the old string was left untouched automatically.
For a very similar problem where we have to deal with array/arrayList instead of String, we need to add Undo logic as well — i.e. Backtracking...

Perform dry run using pen and paper to understand ☺

#pattern

78. Subsets

Recurssion | Backtracking

Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Thought Process

- Initial though will be same i.e. **pick/skip pattern**
- Keep a track on index i through the array, along with the current subset list

Why Arrays/Lists Need "Undo" (Backtracking)

Unlike strings, a `List` in Java is **mutable** (it can be modified directly).

When we write code for an array, we use a **single shared list** called `current` to save memory, rather than creating a new list at every step.

1. `current.add(nums[i]);`
   We add the current number to our list to try the "Include" path.
2. `backtrack(i + 1, nums, current, result);`
   We explore all possibilities down this path.
3. `current.remove(current.size() - 1);`
   This is the undo step (Backtracking). Once the "Include" path is completely finished, we must remove that number from `current` before moving on. Why? Because we need the *exact same* `current` list to be clean and empty of that number when we try the "Exclude" path next.
4. `backtrack(i + 1, nums, current, result);`
   Now we explore the "Exclude" path without that number.

Perform dry run using pen and paper to understand ☺

Dry Run

```java
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(0, nums, new ArrayList<>(), result);
        return result;
    }
private void backtrack(int i, int[] nums, List<Integer> current, List<List<Integer>> result) {
        // Base case: if we've considered all elements
        // add a copy of the current subset
        if (i == nums.length) {
            result.add(new ArrayList<>(current));
            return;
        }
        // Choice 1: Include nums[i]
        current.add(nums[i]);
        backtrack(i + 1, nums, current, result);
        // Backtrack (undo the choice)
        current.remove(current.size() - 1);
        // Choice 2: Exclude nums[i]
        backtrack(i + 1, nums, current, result);
    }
}
```

- **Shared Mutable State:** You pass a single, mutable `ArrayList<Integer> current` down through all recursive branches instead of allocating a brand-new list at every function call.
- **An Explicit Undo Step:** The line `current.remove(current.size() - 1);` directly alters the state back to exactly how it was before you made **Choice 1**.
- **Deep Copying at Leaf Nodes:** Because you are reusing that single `current` list across the entire execution tree, you safely make a snapshot clone (`new ArrayList<>(current)`) when adding it to the `result`. If you didn't do this, every entry in your final `result` list would end up completely empty at the end of the program!

This version is incredibly memory efficient compared to the string version, as it performs minimal object allocation during the tree traversal.

#pattern

46. Permutations

Recurssion | Backtracking

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Input: nums = [1,2,3]
Output:[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Thought Process

1. **In-place Swapping:** By swapping elements at the current index with elements ahead of it, you systematically explore all possible branches of permutations without allocating extra memory for state tracking.
2. **Backtracking Integrity:** The second swap (`swap(i, ind, nums)`) correctly backtracks, restoring the array to its original state before the next iteration of the loop.

Complexity:

- **Time Complexity:** $O(n \cdot n!)$ — There are $n!$ possible permutations for an array of length n, and it takes $O(n)$ time to copy each valid permutation into the result list at the base case. Since generating every permutation is mathematically required, this time complexity is asymptotically optimal.
- **Space Complexity:** $O(n)$ — Beyond the space required to store the output (which is required by the problem), the recursive call stack goes as deep as the length of the array (n), and the swapping is done in-place without needing extra tracking structures like a `visited` array or a separate `path` list.

Here every time we are adding the same array i.e. nums, coz it's values are changing on each call, coz of the sawaping, hence the same same reference is working fine...

```java
class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> ans = new ArrayList<>();
        helper(0, nums, ans);
        return ans;
    }
    void helper(int ind, int[] nums, List<List<Integer>> ans) {
        if(ind == nums.length) {
            List<Integer> val = new ArrayList<>();
            for(int n : nums) {
                val.add(n);
            }
            ans.add(new ArrayList<>(val));
            return;
        }
        for(int i = ind; i < nums.length; i++) {
            swap(i, ind, nums);
            helper(ind + 1, nums, ans);
            swap(i, ind, nums);
        }
    }
    void swap(int x, int y, int[] nums) {
        int temp = nums[x];
        nums[x] = nums[y];
        nums[y] = temp;
    }
}
```

Perform dry run using pen and paper to understand ☺ #pattern

22. Generate Parentheses

Recurssion | Backtracking

Given n pairs of parentheses, write a function to *generate all combinations of well-formed parentheses.*

Input: n = 3
Output: ["((()))","(()())","(())()","()(())","()()()"]

- Backtracking Overview: A recursive technique used to explore all possible combinations to find valid solutions, often involving decision-making at each step.

Dry Run

```java
class Solution {
    public List<String> generateParenthesis(int n) {
        List<String> ans = new ArrayList<>();
        backtrack(ans, "", 0, 0, n);
        return ans;
    }

    public void backtrack(List<String> output, String cur_str, int opn, int cls, int max) {
    if(cur_str.length() == max * 2) {
        output.add(cur_str);
        return;
    }
    if(opn < max) backtrack(output, cur_str + "(", opn + 1, cls, max);
    if(cls < opn) backtrack(output, cur_str + ")", opn, cls + 1, max);
    }
}
```

Thought Process

- Output Container: Initialize an array (e.g., ArrayList) to store all successful combinations.
- Recursive Function: Define a backtrack helper method that passes the current state (current string, counters for open/close, and maximum limit).
- Base Case: A condition to stop recursion. In this problem, it occurs when the current string length equals max * 2, meaning the valid combination is complete and can be added to the output list.
- Decision Logic: At every recursive call, evaluate all possible paths:
  - Option 1 (Add Opening Parenthesis): Only if the count of open parentheses is less than the max allowed.
  - Option 2 (Add Closing Parenthesis): Only if the count of closed parentheses is less than the count of open (to ensure well-formedness).

Perform dry run using pen and paper to understand ☺ #pattern

39. Combination Sum

Recurssion | Backtracking

Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]

Thought Process

- Pick or Skip approach with some additional checks
  - if target is -ve or ind == length of array, then return
- For each recursive call we can either pick the element or leave it
  - If picked, then we should be reducing it from target
  - In the other choice simply have to increase the ind

Dry Run

```java
class Solution {
    public List<List<Integer>> combinationSum(int[] cnd, int tg) {
        List<List<Integer>> ans = new ArrayList<>();
        helper(0, cnd, tg, ans, new ArrayList<>());
        return ans;
    }
    void helper(int ind, int[] cnd, int tg, List<List<Integer>> ans, List<Integer> val) {
        if (tg == 0) {
            ans.add(new ArrayList<>(val));
            return;
        }
        if (tg < 0 || ind == cnd.length) {
            return;
        }
        val.add(cnd[ind]);
        helper(ind, cnd, tg - cnd[ind], ans, val);
        val.remove(val.size() - 1);
        helper(ind + 1, cnd, tg, ans, val);
    }
}
```

Perform dry run using pen and paper to understand ☺

#pattern

206. Reverse Linked List

Two Pointers | Linked List

Approach 1 *#itrative*

→ Initialize pointer temp & prev
→ Track current node with temp & previous with prev
→ Iterate while temp is not null
  → store next node in front
  → point temp.next to prev to reverse the link
  → shift prev to temp and temp to front
→ when iteration finishes return prev as the new head

Time Complexity → O(n)
Space Complexity → O(1)

```
11  class Solution {
12      public ListNode reverseList(ListNode head) {
13          ListNode temp = head, prev = null;
14          while(temp != null) {
15              ListNode front = temp.next;
16              temp.next = prev;
17              prev = temp;
18              temp = front;
19          }
20          return prev;
21      }
22  }
```

#pattern

206. Reverse Linked List

Recursion | Linked List

Approach 2 *#recurssive*

→ Base Case: Check if head is null or head.next is null. If so, return head as the base head of the reversed list.

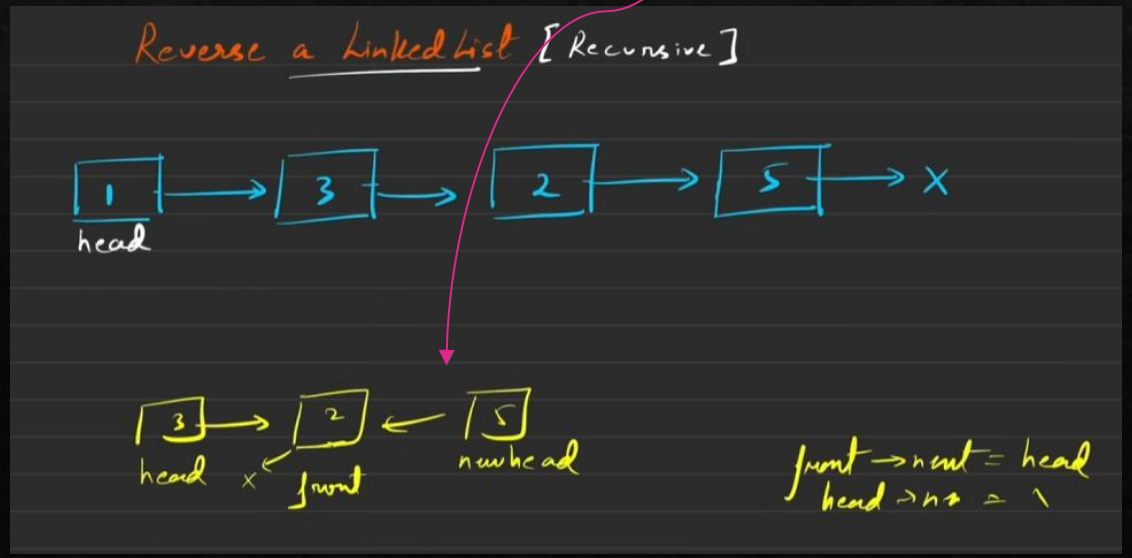
→ Recursive Call: Recursively call reverseList(head.next) to reverse the rest of the list, returning the new head of the fully reversed sub-list (newHead).

→ Reverse Connection: Set front to head.next (the node directly after head). Point front.next back to head to reverse the pointer direction.

→ Break Old Link: Set head.next to null to avoid creating a cycle in the list.

→ Return: Return newHead up the recursion chain

```
11  class Solution {
12      public ListNode reverseList(ListNode head) {
13          if(head == null || head.next == null) return head;
14          ListNode newHead = reverseList(head.next);
15          ListNode front = head.next;
16          front.next = head;
17          head.next = null;
18          return newHead;
19      }
20  }
```



Time Complexity → $O(n)$
Space Complexity → $O(n)$

The key point in here is swaping → from front to head thing

#pattern

21. Merge Two Sorted Lists

Two Pointers | Linked List

Approach 1

→ ans points to a dummy node with value -1
→ temp points to ans
→ curr1 points to the head of list1, and curr2 points to the head of list2
→ Traverse until curr1 && curr2 are not null
  → Comapre value of curr1 & curr2, whichever is less add that into temp.next and move curr pointer to curr.next of respectively
  → Move temp to temp.next as well
→ Check if any curr pointer are not null then add into temp
→ Return ans.next at the end

Time Complexity → $O(n + m)$
Space Complexity → $O(1)$

```java
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode ans = new ListNode(-1);
        ListNode temp = ans;
        ListNode curr1 = list1, curr2 = list2;
        while(curr1 != null && curr2 != null) {
            if (curr1.val < curr2.val) {
                temp.next = curr1;
                curr1 = curr1.next;
            } else {
                temp.next = curr2;
                curr2 = curr2.next;
            }
            temp = temp.next;
        }
        if(curr1 != null) temp.next = curr1;
        if(curr2 != null) temp.next = curr2;
        return ans.next;
    }
}
```

#pattern

141. Linked List Cycle

Two Pointers | Linked List

Approach 1

→ If head == null || head.next null then return false
→ Initialize pointer slow & fast
→ Iterate while fast & fast.next is not null
  → Slow moves one at a time but fast moves two steps at a time
  → At any point of time if slow == fast return true
→ return false in the end

Time Complexity → O(n)
Space Complexity → O(1)

```java
public class Solution {
    public boolean hasCycle(ListNode head) {
        if(head == null || head.next == null) return false;

        ListNode slow = head;
        ListNode fast = head;
        while(fast != null && fast.next != null){
            slow = slow.next;
            fast = fast.next.next;

            if(slow == fast) return true;
        }
        return false;
    }
}
```

To understand why `slow` and `fast` must meet in a cycle, imagine two runners moving around a circular track.

- **Slow runner (`slow`):** Moves **1 step** per turn.
- **Fast runner (`fast`):** Moves **2 steps** per turn.

---

1. Entering the Cycle
Before the fast pointer reaches the end of the list, either it finds `null` (no cycle → returns `false`) or both pointers eventually enter the cyclic loop.

2. Relative Speed inside the Cycle
Once both pointers are inside the loop, focus on the distance between them:

- In every step of the `while` loop, `fast` moves 2 steps forward while `slow` moves 1 step forward.
- The gap between them decreases by $2 - 1 = 1$ **step** in every iteration.

Because the gap shrinks by exactly **1 node per step**, `fast` is guaranteed to catch up to `slow` without ever "jumping over" it.

Step-by-Step Example

Consider a list with a cycle: `1 -> 2 -> 3 -> 4 -> 2` (node `4` points back to node `2`).

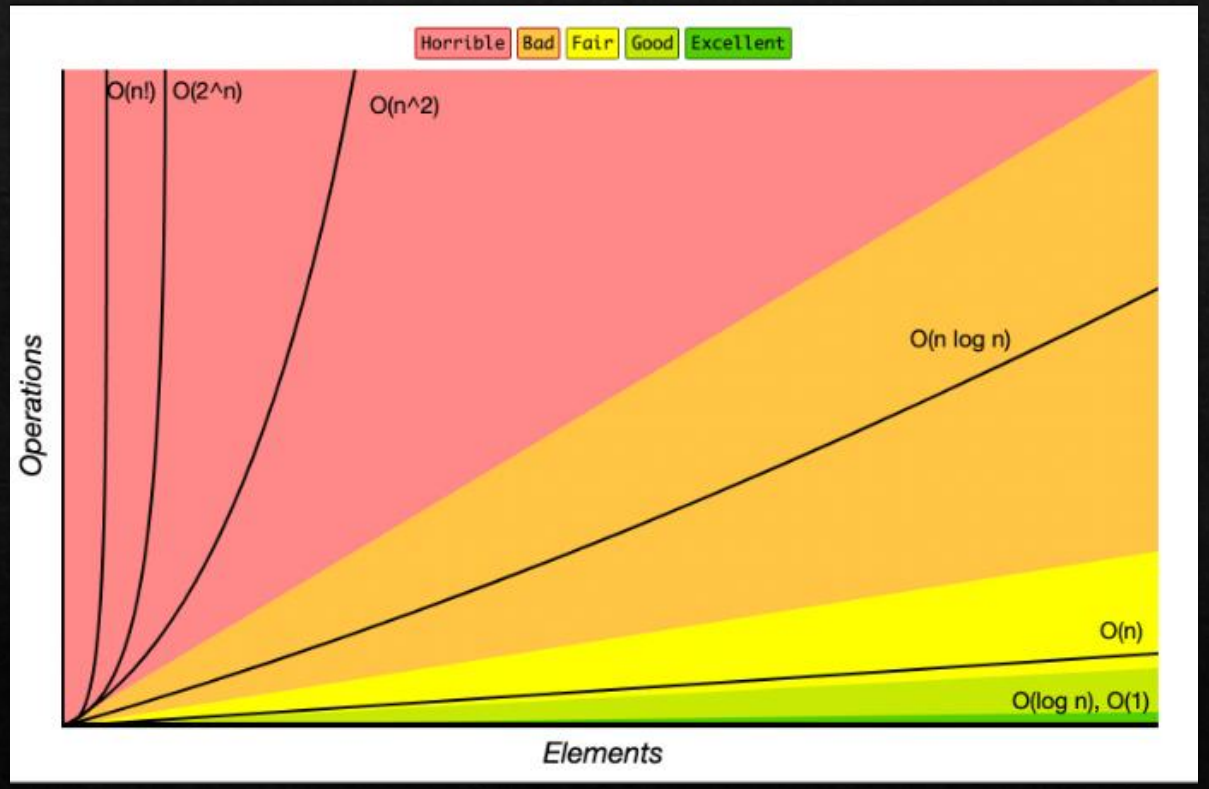
Iteration	`slow` position	`fast` position
Start	Node 1	Node 1
Step 1	Node 2	Node 3
Step 2	Node 3	Node 2 (loop back)
Step 3	Node 4	Node 4

Export to Sheets

Since the gap decreases by 1 on every iteration, `fast` will always close the distance and equal `slow` in a finite number of steps if a cycle exists.

#pattern

BigO Graph & Time Complexity Cheat Sheet



Algorithm	Best Time Complexity	Average Time Complexity	Worst Time Complexity	Worst Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Quick Sort	O(nlogn)	O(nlogn)	O(n^2)	O(log n)
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)
Tim Sort	O(n)	O(nlogn)	O(nlogn)	O(n)
Shell Sort	O(n)	O((nlog(n))^2)	O((nlog(n))^2)	O(1)

Pattern Recognition

- Array/String
- Linked List Patterns
- 8 Most Imp Patterns
- FAQs

HAPPY GRINDING

Quick Notes

- Most useful Character comparison method
- All About String method
- All StringBuilder methods
- Must known Arrays methods

HAPPY GRINDING

Resources

- strivers-a2z-sheet for problems
- leetcode for problem solving
- programiz for online-compiler
- pythontutor for dry run
- ultrashort for URL shorten
- ray for code snippet (Theme: raindrop)
- chatgpt for content
- DSA Pattern Sheet for blind test

HAPPY GRINDING